



Capital University [formerly Helwan University]

Faculty of Computing and Artificial Intelligence

Artificial Intelligence Department

VISUALine: A Modular System for Local AI Video Processing on Consumer GPUs

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Computing and Artificial Intelligence at the
Artificial Intelligence Department, the Faculty of Computing and
Artificial Intelligence, Capital University

Supervisor Name

Dr. Islam Gamal

A graduation project dissertation by:

Mohammed Marzouk 20220416

Karem Moamen 20220348

Mazen Abdelaal 20220359

Tadros Milad 20220122

Yahia Ragab 20220556

Mohamed Ezzat 20220417

June, 2026



جامعة العاصمة (جامعة حلوان سابقاً)

كلية الحاسبات والذكاء الاصطناعي

قسم الذكاء الاصطناعي

نظام نموذجي لمعالجة الفيديو بالذكاء الاصطناعي محلياً على معالجات الرسومات الاستهلاكية

رسالة مقدمة ضمن متطلبات الحصول على درجة البكالوريوس في الحاسبات والذكاء الاصطناعي،
بقسم الذكاء الاصطناعي كلية الحاسبات والذكاء الاصطناعي، جامعة العاصمة

اسم المشرف
د. إسلام جمال

رسالة مشروع تخرج مقدمة من:

٢٠٢٢٠٤١٦ محمد مرزوق

٢٠٢٢٠٣٤٨ كريم مؤمن

٢٠٢٢٠٣٥٩ مازن عبد العال

٢٠٢٢٠١٢٢ تادرس ميلاد

٢٠٢٢٠٥٥٦ يحيى رجب

٢٠٢٢٠٤١٧ محمد عزت

يونيو ٢٠٢٦

Abstract

VISUALine is a modular, production-oriented framework for automated visual enhancement and video editing. The project addresses the challenge of combining multiple specialized computer-vision models into reproducible end-to-end processing workflows while containing runtime overhead and GPU memory pressure. Motivation arises from the high cost and complexity of manual multimedia editing and from practical constraints when deploying heterogeneous deep models on limited hardware.

Methodologically, VISUALine adopts a node-based pipeline abstraction: discrete processing stages are encapsulated as reusable nodes that consume and emit tensor payloads, enabling composition of tasks such as semantic detection, mask-guided editing, re-framing, frame interpolation, and super-resolution. A lightweight execution engine compiles sequential execution plans and runs them under inference mode, while a resource manager implements an LRU model cache with VRAM soft limits and controlled eviction to minimize repeated model loads. Mixed-precision inference and GPU acceleration are used where available to improve throughput and reduce memory usage.

The implementation integrates pre-trained neural modules covering temporal interpolation, image upscaling, segmentation, and text-prompted detection, and relies on established numerical and media libraries for efficient tensor computation and robust input/output handling. The system accepts heterogeneous media inputs (images and videos) and exposes both programmatic and interactive interfaces for experimentation and deployment.

Keywords: local AI, video processing, computer vision systems, consumer GPU, modular pipelines, prompt-based editing, video enhancement, desktop AI application

الملخص

يعتبر نظام فيجوالاين إطار عمل نموذجيًا وموجهًا للإنتاج ومخصصًا لتحسين البصري المؤتمت وتحرير الفيديو. يعالج هذا المشروع تحدي دمج نماذج رؤية حاسوبية متعددة ومتخصصة في تدفقات عمل معالجة شاملة وقابلة للتكرار، مع الحد من العبء الإضافي لوقت التشغيل وضغط ذاكرة وحدة معالجة الرسوميات. ينبع دافع المشروع من التكلفة العالية والتعقيد المرتبطين بالتحرير اليدوي للوسائط المتعددة، بالإضافة إلى القيود العملية عند نشر نماذج عميقة غير متجانسة على أجهزة ذات قدرات محدودة.

من الناحية المنهجية، يعتمد النظام على تجريد خطوط المعالجة القائمة على العقد، حيث يتم تغليف مراحل المعالجة المنفصلة كعقد قابلة لإعادة الاستخدام تستهلك وتنتج حمولات مخططات مصفوفات التنس، مما يتيح تكوين مهام مثل الكشف الدلالي، والتحرير الموجه بالأقنعة، وإعادة التأطير، واستيفاء الإطارات، والدقة الفائقة. يقوم محرك تنفيذ خفيف الوزن بتجميع خطط التنفيذ المتسلسلة وتشغيلها تحت وضع الاستدلال، بينما يقوم مدير الموارد بتنفيذ ذاكرة تخزين مؤقت للنماذج تعتمد على خوارزمية الأقل استخدامًا مؤخرًا، مع وضع حدود مرنة لذاكرة الفيديو العشوائية وإخلاء محكوم للموارد لتقليل عمليات تحميل النماذج المتكررة. كما يُستخدم استدلال الدقة المختلطة وتسريع وحدة معالجة الرسوميات لتحسين معدل الإنتاجية وتقليل استهلاك الذاكرة.

يتكامل التنفيذ الفعلي للنظام مع وحدات عصبية مدربة مسبقًا تغطي الاستيفاء الزمني، وترقية حجم الصور، والتجزئة، والكشف الموجه بالنصوص، ويعتمد على المكتبات العددية ومكتبات الوسائط الراسخة لإجراء حسابات المصفوفات الفعالة ومعالجة المدخلات والمخرجات بشكل قوي. يقبل النظام مدخلات وسائط غير متجانسة من صور وفيديوهات، ويوفر واجهات برمجية وتفاعلية للتجريب والنشر.

Acknowledgments

We would like to express our sincere gratitude to all those who contributed to the successful completion of this project. We are particularly indebted to our project supervisor for their invaluable guidance, constructive feedback, and continuous support throughout the design, implementation, and evaluation phases. Their expertise in computer vision and deep learning frameworks proved instrumental in navigating the technical complexities inherent to this work.

We also extend our appreciation to the faculty members of the Computer Science department who provided essential courses and seminars in machine learning, software architecture, and systems optimization. Their foundational knowledge enriched our understanding of the theoretical and practical aspects of neural networks and GPU-accelerated computing.

This project would not have been possible without the collaborative efforts of our team members. We acknowledge the significant contributions of Mohammed Marzouk, Tadros Milad, Yahia Ragab, Mazen Abdelaal, Karem Moamen, and Mohamed Ezzat, each of whom brought specialized skills and dedication to different components of VISUALine. Their collective effort in model integration, pipeline orchestration, resource management, and interface development was essential to realizing this comprehensive visual processing framework.

Dedication

We dedicate this work to our families, whose steadfast support and encouragement have been the foundation of our academic journey. We also dedicate VISUALine to the collaborative spirit of open-source development and to the advancement of practical, production-grade artificial intelligence systems that make powerful visual technologies accessible to researchers and practitioners alike.

Contents

Abstract	I
Acknowledgments	III
List of Figures	VIII
List of Tables	X
1 Introduction	1
1.1 Overview	1
1.2 Motivation	2
1.3 Problem Statement	3
1.3.1 Engineering Challenges	3
1.3.2 Research-Oriented Questions	4
1.4 Objectives	5
2 Literature Review	6
2.1 Introduction	6
2.2 GroundingDINO	7
2.2.1 Overview and Motivation	7
2.2.2 Architecture and Method	8

2.3	Segment Anything Model 2 (SAM 2)	11
2.3.1	Overview and Motivation	11
2.3.2	Architecture and Method	13
2.4	RIFE Model (Real-Time Intermediate Flow Estimation)	15
2.4.1	Overview and Motivation	15
2.4.2	Architecture and Method	17
2.5	SPAN Model (Swift Parameter-free Attention Network)	19
2.5.1	Overview and Motivation	19
2.5.2	Architecture and Method	21
3	Methodologies	24
3.1	Introduction	24
3.2	Pipeline Design	25
3.2.1	Node Class Resolution and Instantiation	26
3.2.2	Control-to-Node Binding Mechanism	26
3.2.3	Multi-Level Configuration Hierarchy	27
3.2.4	Workflow Variant Pattern	27
3.3	Workflow Library	27
3.3.1	Cinematic Prompt Blur – Fast	28
3.3.2	Cinematic Prompt Blur – HQ SAM2	28
3.3.3	Privacy Redaction	28
3.3.4	Old Video Enhancement – Fast	28
3.3.5	Old Video Enhancement – SPAN x4	28
3.3.6	Smart Vertical Reframe	29
3.3.7	Slow Motion RIFE x2	29
3.4	Dynamic Upscaling Overview	29

3.4.1	What is Dynamic Upscaling?	29
3.4.2	Problem Solved	29
3.4.3	Core Insight	30
3.4.4	Dynamic Approach	30
3.4.5	Benefits	30
3.5	Node Interface and Data Flow	30
3.5.1	Node Lifecycle Contract	31
3.5.2	Shared Data Convention	32
3.5.3	Mask Channel Semantics	32
3.5.4	Rationale for a Fixed Tensor Convention	33
3.6	Execution Runtime	33
3.6.1	PipelineManager	34
3.6.2	TaskExecuter	35
3.6.3	ResourceManager	36
3.6.4	VideoProcessor	36
3.7	User-Facing Application	38
3.7.1	Backend API Architecture	38
3.7.2	Dynamic Interface Rendering	39
3.7.3	Workflow Selection and Metadata Display	39
3.7.4	Local Preview and Output Management	40
4	System Analysis and Design	41
4.1	Introduction	41
4.2	System Software and Hardware Requirements	42
4.2.1	Software Requirements	42
4.2.2	Hardware Requirements	43

4.2.3	Performance-Related Notes	43
4.2.4	Compatibility Constraints	44
4.3	Assumptions and Dependencies	44
4.4	Diagrams	45
4.4.1	System Architecture Diagram	45
4.4.2	Pipeline Sequence Diagrams	46
4.4.3	Privacy Redaction Pipeline Sequence Diagrams	52
5	Results and Experiments	58
5.1	Experimental Evaluation Plan	58
5.1.1	Hardware	58
5.1.2	Dataset	58
5.1.3	Runtime Metrics	59
5.1.4	Fast vs High-quality Blur Comparison	60
5.1.5	Enhancement Quality	60
5.1.6	Redaction Reliability	61
6	Discussion and Conclusion	62
6.1	Discussion	62
6.2	Limitations	63
6.3	Conclusion	63
	References	65

List of Figures

1	The Framework of GroundingDINO	9
2	Structural schema of the SAM 2 architecture	13
3	Pipeline Architecture of RIFE	17
4	Pipeline Architecture of SPAN	21
5	The 7-layer system architecture of VISUALine.	45
6	Sequence diagram illustrating API startup, parameters rendering, job queuing, and WebSocket progress updates.	47
7	Sequence diagram showing configuration loading, dynamic node setup, and resource allocation.	48
8	Sequence diagram detailing prompt-based bounding-box detection, filtering, and caching optimizations (Part 1: Data ingestion and pipeline triggers; Part 2: Bounding box inference and caching).	49
9	Sequence diagram for SAM2 temporal frame propagation, mask generation, and file cleanup.	50
10	Sequence diagram showing image/video blending, frame output serialization, and OOM recovery handling.	51
11	Sequence diagram showing privacy pipeline configuration loading, dynamic two-node setup, and resource allocation.	53
12	Sequence diagram detailing prompt-based bounding-box detection, filtering, and caching optimizations.	54

-
- 13 Sequence diagram showing region redaction modes (blur, pixelate, black/white) applied to detected areas. 56
 - 14 Sequence diagram showing frame enqueueing, FFmpeg pipe writing, and audio muxing for the final video output. 57

List of Tables

5.1	Hardware and software environment.	59
5.2	Benchmark media set.	59
5.3	End-to-end runtime benchmark results on RTX 3050.	60
5.4	Comparison between Cinematic Blur Fast and HQ SAM2.	60
5.5	Enhancement quality benchmark (estimated ranges).	61
5.6	Privacy redaction reliability on Clip V1.	61

Chapter 1

Introduction

1.1 Overview

Video is everywhere: social clips, family archives, lecture recordings, and footage that was never meant for a professional edit suite. Yet many edits people actually want, such as a blurred background behind a subject, faces blurred for privacy, sharper old footage, a vertical crop that follows someone walking, or smooth slow motion, still mean either hours in a non-linear editor or sending files to a cloud service. That second path brings subscriptions, upload time, and a real privacy cost when the material is personal or sensitive.

VISUALine is an open source, locally run suite for AI-assisted visual enhancement. The goal is practical: upscale, restore, interpolate frames, segment objects from text prompts, and composite effects, all on the user's machine, without shipping media to remote servers. The stack is built in Python 3.11 and uses GPU acceleration when CUDA is available.

The design is deliberately layered. At the top, workflows are declared in YAML: each file describes a named pipeline (for example, fast cinematic blur versus a higher quality variant that uses SAM 2). Below that sit composable processing nodes, each responsible for one transformation, such as detection, masking, blur, super resolution, interpolation, and similar steps. A runtime engine loads those definitions, sets up every node before inference starts, runs batches on GPU tensors, and enforces memory policy so several large models can be chained without exhausting VRAM. Finally, client interfaces expose the same workflows: a local HTTP API for the desktop application, an Electron Vue front end for day to day use.

From the user’s perspective, the flow is straightforward. They pick a workflow, such as Cinematic Prompt Blur (Fast), Old Video Enhancement (SPAN x4), or Slow Motion RIFE x2, then adjust the sliders and prompts defined for that pipeline, and submit an image or video job. The backend queues the work, applies any runtime overrides to the active configuration, runs the compiled sequence of nodes, and returns processed media with optional progress updates.

For video, frames are streamed through FFmpeg-based readers and writers; audio is copied back into the output when the source has a soundtrack. Common container and still-image formats are supported. What matters for the thesis is not a list of extensions but the intent: advanced vision models are packaged as repeatable workflows, not one-off scripts, so creators and privacy-conscious users can get production-oriented results while keeping data and compute under their own control.

1.2 Motivation

Several concrete pressures shaped VISUALine, and each maps to a design choice in the implementation.

- Reliance on cloud video tools. Many commercial products require upload, a paid plan, and a stable connection. VISUALine is built for local-first use: the API listens only on the loopback interface, cross-origin access is limited to local clients, and outputs stay on disk under the project’s data area. That model fits sensitive footage, offline editing, and graduation-scale deployment without a hosted backend.
- Scattered open-source models. Strong components (including GroundingDINO for open-vocabulary detection, SAM 2 for segmentation, SPAN for super-resolution, and RIFE for interpolation) normally live in separate projects with different APIs. Wiring them together for every new idea is tedious. VISUALine wraps architectures behind a single node contract so new capabilities are added as pipeline steps rather than as disconnected experiments.
- Rigid general-purpose editors. Effects like “blur the background but keep the person described in a text prompt sharp” or “reframe to 9:16 while following a subject” chain detection, masking, and compositing. Rebuilding that by hand for each clip does not scale. The project encodes these chains as named pipelines with UI metadata (labels, categories, speed hints), so the desktop client and the playground invoke the same logic without rewriting application code.

- GPU memory when chaining models. Loading several large networks in one session can exhaust VRAM. A shared resource manager caches models with LRU eviction against a soft limit (by default roughly 85% of available GPU memory), blocks mid-run loading so initialization happens up front during setup, and the pipeline runner can split batches after out-of-memory errors. That directly targets a barrier that otherwise makes multi-stage local inference impractical on consumer hardware.
- Need to validate workflows quickly. During development, rebuilding a desktop client for every parameter change is slow. The Streamlit playground loads the same pipeline manager, reuses an already-loaded pipeline when the configuration has not changed, and shows before-and-after previews, which are enough to tune YAML and node parameters iteratively.

Taken together, these motivations match the project's direction: an AI-powered visual enhancement suite with a usable interface and room to add new nodes and pipelines without rewriting the core engine.

1.3 Problem Statement

The core question VISUALine addresses can be stated as follows:

How can non-expert users and student developers run complex, multi-model video and image enhancement workflows on their own hardware, with adjustable parameters, tolerable GPU usage, and a single application experience, without proprietary cloud pipelines or a new glue script for every task?

That broad question breaks into engineering and research challenges that the current system is built to explore.

1.3.1 Engineering Challenges

1. Workflow specification and reuse. Real edits are multi-step: detect, then mask, then blur; or pre-enhance, then upscale, then sharpen. Without a standard way to compose steps, every effect becomes a monolithic script. The system must represent workflows declaratively and run them reliably on single images and on long videos.

2. Temporal consistency and throughput. Running detection independently on every frame causes flicker and wastes compute. Detection nodes therefore support running every n frames, reusing the last bounding boxes, and tolerating short gaps when the subject disappears. Interpolation nodes need frame history and can double the effective frame rate. For video, the engine uses threaded read and write paths, temporal windows where needed, and batching so streaming is a first-class concern rather than an afterthought.
3. Heterogeneous data between nodes. Some steps expect plain image tensors; others pass structured payloads that carry both pixels and detection geometry. The executor runs a linear plan and must tolerate those mixed contracts so downstream nodes can apply masks and crops consistently.
4. Hardware variability. Machines differ in VRAM, encoder support, and whether CUDA, Apple MPS, or CPU is available. The stack should fall back gracefully (using half precision where supported, tiled super-resolution when full-frame inference does not fit, and smaller batches after CUDA out-of-memory) instead of failing opaquely.
5. Operability for end users. Technical pipelines are useless if parameters stay buried in configuration files. Each production workflow exposes controls mapped to specific nodes and parameters; the API merges user choices into the active run configuration so the desktop UI can drive jobs without manual editing.

1.3.2 Research-Oriented Questions

- Can YAML-orchestrated, sequential node pipelines express enough real editing scenarios (including blur, privacy redaction, restoration, reframing, and slow motion) while staying maintainable as new models appear?
- Does centralized VRAM management make it practical to chain large vision models on consumer GPUs?
- Does offering both fast workflows (box-based masks, lighter enhancement) and heavy workflows (SAM 2 segmentation, SPAN $\times 4$ super-resolution) give users a usable speed-quality trade-off locally?

A working hypothesis behind the current design is that a thin sequential executor, strongly scoped nodes, and explicit memory policy are enough for the implemented workflows, without a full graph scheduler or cloud offload, but at the cost of limited parallelism and no dynamic branching inside a single pipeline.

1.4 Objectives

The primary objectives of VISUALine, as reflected in what is already implemented, are:

1. Modular inference engine. Provide a local framework where new capabilities are added as node implementations registered through YAML, without changing the core executor.
2. Production-oriented workflows. Ship end-user workflows including:
 - Cinematic and prompt-guided background blur (fast box-mask paths and higher-quality SAM 2 variants).
 - Privacy redaction driven by open-vocabulary detection, with blur, pixelation, or blackout styles.
 - Old or low-quality footage enhancement (a faster classical chain and a SPAN $\times 4$ super-resolution chain).
 - Smart vertical reframing for social aspect ratios with subject tracking.
 - Slow-motion generation using RIFE-style $\times 2$ frame interpolation.
3. Unified media I/O. Handle still images (single files and batched folders grouped by resolution) and videos through FFmpeg pipes, with sensible limits on processing resolution and audio preserved when possible.
4. Stable application API. Expose HTTP endpoints for listing pipelines, submitting image and video jobs, tracking asynchronous work, and reporting system health, which allows the Electron client to drive the backend without embedding Python.
5. Local asset management. Resolve model weights and exported checkpoints on the machine, integrating vendored and dependency-pinned vision stacks so runs are reproducible offline after initial setup.

Chapter 2

Literature Review

2.1 Introduction

Modern AI-assisted video editing depends on a stack of specialized deep learning models rather than a single general-purpose network. Restoration requires recovering spatial detail from degraded frames; temporal editing requires synthesizing plausible intermediate motion; and semantic editing—such as background blur, privacy masking, or subject-driven reframing—requires knowing precisely what to keep sharp and what to alter. Each task has evolved its own research line, benchmark culture, and deployment constraints. A practical local editing suite must therefore integrate several state-of-the-art components, each chosen for a distinct role in the processing chain.

This chapter reviews the four primary model families that underpin VISUALine’s implemented workflows:

- **GroundingDINO**: Used for open-vocabulary visual object detection via text prompts.
- **SAM 2** (Segment Anything Model 2): Utilized for dense spatio-temporal video object segmentation.
- **SPAN** (Swift Parameter-free Attention Network): Deployed for single-image and frame-by-frame super-resolution.
- **RIFE** (Real-Time Intermediate Flow Estimation): Leveraged for learned frame interpolation and temporal smoothing.

Together, these architectures cover the perception-to-output pipeline that the project runs in production configurations: open-vocabulary detection, dense spatio-temporal segmentation, single-image super-resolution, and learned frame interpolation.

The selection of these models is not arbitrary; it strictly mirrors the node graphs declared in the project’s YAML pipeline configurations. For instance, GroundingDINO and SAM [2] appear in cinematic blur, privacy redaction, and smart vertical reframe workflows, where text prompts or bounding boxes must be turned into reliable spatial masks over time. Conversely, SPAN powers the high-quality old-video enhancement pipeline as a $4\times$ super-resolution stage following classical pre-processing operations, while RIFE drives the slow-motion workflow by doubling the effective frame rate through intermediate frame synthesis.

Understanding these models at the literature level is a prerequisite to explaining why VISUALine composes them sequentially, how VRAM and latency trade-offs arise, and where fast approximations (e.g., bounding box masks, sparse detection) differ from heavy, high-quality computation paths (e.g., dense SAM [2] tracking masks, tiled SPAN inference).

2.2 GroundingDINO

2.2.1 Overview and Motivation

Overview

GroundingDINO is an advanced open-set object detector developed by marrying the Transformer-based detector DINO with grounded pre-training, which can detect arbitrary objects with human inputs such as category names or referring expressions. The key solution of open-set object detection is introducing language to a closed-set detector for open-set concept generalization. To effectively fuse language and vision modalities, GroundingDINO conceptually divides a closed-set detector into three phases and proposes a tight fusion solution, which includes a feature enhancer, a language-guided query selection, and a cross-modality decoder for modalities fusion.

The model is a dual-encoder-single-decoder architecture. It contains an image

backbone for image feature extraction, a text backbone for text feature extraction, a feature enhancer for image and text feature fusion, a language-guided query selection module for query initialization, and a cross-modality decoder for box refinement. GroundingDINO outputs multiple pairs of object boxes and noun phrases for a given (Image, Text) pair. Both object detection and Referring Expression Comprehension (REC) tasks can be aligned with this pipeline.

Motivation

The development of GroundingDINO is driven by two main principles to improve open-set scenarios:

- **Tight Modality Fusion Based on DINO:** The key to open-set detection is introducing language for unseen object generalization. Most existing open-set detectors are developed by extending closed-set detectors to open-set scenarios with language information. A closed-set detector typically has three important modules: a backbone for feature extraction, a neck for feature enhancement, and a head for region refinement (or box prediction). Previous works only fuse multi-modal information in partial phases, which may lead to sub-optimal language generalization ability. The layer-by-layer design of Transformer-based detectors like DINO enables them to interact with language information easily. Under this principle, GroundingDINO designs

2.2.2 Architecture and Method

Architectural Overview

GroundingDINO is a dual-encoder-single-decoder architecture. It contains an image backbone for image feature extraction, a text backbone for text feature extraction, a feature enhancer for image and text feature fusion, a language-guided query selection module for query initialization, and a cross-modality decoder for box refinement.

CHAPTER 2. LITERATURE REVIEW

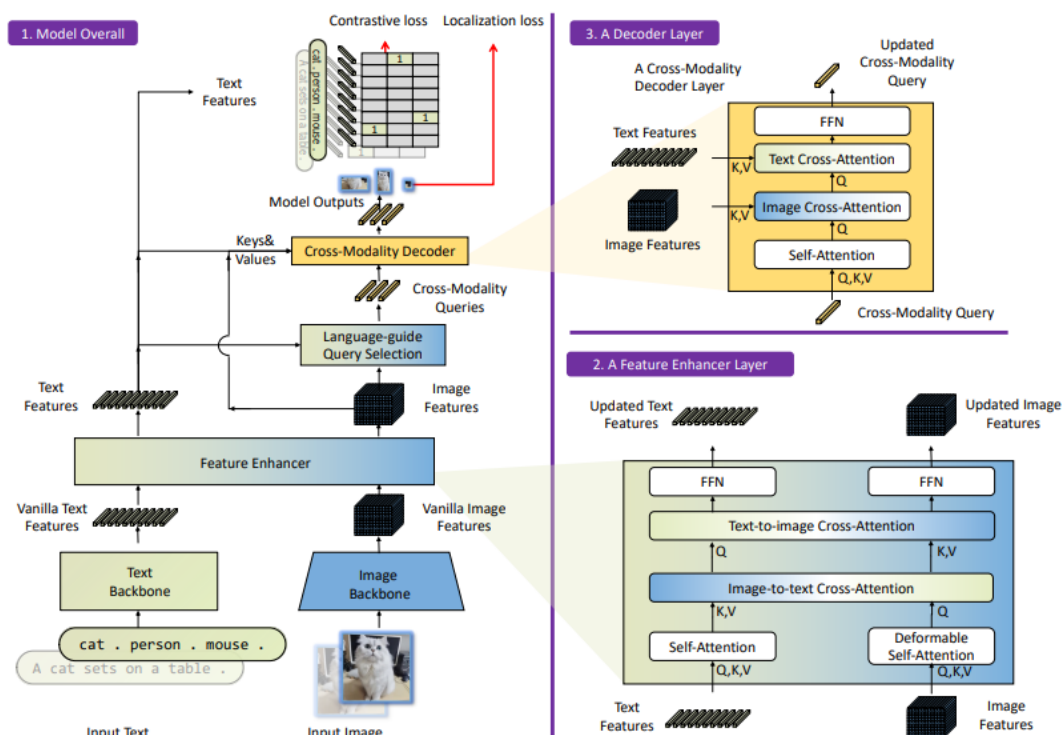


Figure 1: The framework of Grounding DINO. We present the overall framework, a feature enhancer layer, and a decoder layer in block 1, block 2, and block 3, respectively.

GroundingDINO outputs multiple pairs of object boxes and noun phrases for a given (Image, Text) pair. As shown in Fig. 1, the model locates objects from the input image and extracts the corresponding words from the input text as labels. Both object detection and Referring Expression Comprehension (REC) tasks can be aligned with this pipeline. Following GLIP, all category names are concatenated as input texts for object detection tasks. REC utilizes the output object with the largest score as the result for each text input.

For each (Image, Text) pair, vanilla image and text features are first extracted by their respective backbones and fed into a feature enhancer for cross-modality fusion. A language-guided query selection module then selects the most text-relevant image features as decoder queries. These queries are fed into a cross-modality decoder, which probes both modal features and updates the queries. The output queries of the last decoder layer are used to predict object boxes and extract corresponding phrases.

Feature Extraction and Enhancer

Given an (Image, Text) pair, multi-scale image features are extracted with an image backbone such as Swin Transformer, and text features with a text backbone such as BERT. After extracting these vanilla features, they are fed into a feature enhancer for cross-modality fusion.

The feature enhancer stacks multiple enhancer layers, each applying deformable self-attention to image features and vanilla self-attention to text features. Inspired by GLIP, image-to-text and text-to-image cross-attention modules are added within each layer to align features across modalities.

Language-Guided Query Selection

To guide detection with the input text, a language-guided query selection module selects the image features most relevant to the text prompt as decoder queries. This is achieved by scoring each image token against all text tokens via cross-modal similarity, then selecting the top N_q highest-scoring image features. Following DINO, each selected query has a content part (learnable) and a positional part (dynamic anchor boxes initialized from encoder outputs).

Cross-Modality Decoder

The cross-modality decoder refines the selected queries by alternating between image and text cross-attention at each layer. Compared to the standard DINO decoder, each layer includes an additional text cross-attention step, which continuously injects language information into the queries throughout decoding for stronger modality alignment.

Sub-Sentence Level Text Feature

When multiple category names are passed as a concatenated text prompt, word-level attention can introduce unwanted dependencies between unrelated categories. To prevent this, GroundingDINO applies attention masks that block interactions between separate category names — a design referred to as sub-sentence level representation. This preserves per-word features for fine-grained understanding while eliminating cross-category interference, which is directly relevant to how the model behaves in VISUALine’s multi-label detection prompts.

2.3 Segment Anything Model 2 (SAM 2)

2.3.1 Overview and Motivation

Overview

Segment Anything (SA) introduced a foundation model for promptable segmentation in images. However, an image is only a static snapshot of the real world in which visual segments can exhibit complex motion, and a significant portion of multimedia content is now recorded with a temporal dimension, particularly in video data. Many important applications require temporal localization beyond image-level segmentation. The Segment Anything Model 2 (SAM 2) is a unified model for video and image segmentation, considering an image as a single-frame video.

SAM 2 focuses on the Promptable Visual Segmentation (PVS) task that generalizes image segmentation to the video domain. The task takes as input points, boxes, or masks on any frame of the video to define a segment of interest for which the spatio-temporal mask (i.e., a ‘masklet’) is to be predicted. Once a masklet is predicted, it can be iteratively refined by providing prompts in additional frames.

SAM 2 is equipped with a memory that stores information about the object and previous interactions, which allows it to generate masklet predictions throughout the video, and also effectively correct these based on the stored memory context of the object from previously observed frames. SAM 2 is capable of interactively segmenting regions through prompts (clicks, boxes, or masks) on one or multiple

video frames by utilizing a streaming memory that stores previous prompts and predictions.

Motivation

The development of SAM 2 is driven by several key challenges and limitations in existing visual segmentation frameworks:

- **Spatio-Temporal Complexity and Unique Video Challenges:** Segmentation in video aims to determine the spatio-temporal extent of entities, which presents unique challenges beyond those in images. Entities can undergo significant changes in appearance due to motion, deformation, occlusion, lighting changes, and other factors. Videos often have lower quality than images due to camera motion, blur, and lower resolution. Further, efficient processing of a large number of frames is a key challenge. While SA successfully addresses segmentation in images, existing video segmentation models fall short in providing a comparable capability to “segment anything in videos.”
- **Limitations of Decoupled SAM + Tracker Approaches:** Recent interactive video object segmentation (iVOS) works have used a combination of SAM on images with video trackers based on masks or points. However, these approaches have limitations: the tracker may not work for all objects, SAM may not perform well on video frames, and there is no mechanism to interactively refine a model’s mistakes, other than re-annotating using SAM in each frame and restarting the tracking from there. In contrast, SAM 2’s streaming architecture processes video frames one at a time, equipped with a memory attention module to attend to previous memories of the target object, allowing a single click to recover lost tracking rather than restarting from scratch.
- **Semantic and Scale Constraints in Video Datasets:** Current video segmentation datasets lack sufficient coverage to achieve the capability of “segmenting anything in videos.” Their annotations typically cover entire objects (not parts) and datasets are often centered around specific object classes, such as people, vehicles, and animals. To mitigate this data limitation, a data engine is deployed to generate the Segment Anything Video (SA-V) dataset. This data engine is not restricted to objects of specific categories, but instead targeted to provide training data for segmenting any object with a valid boundary, including parts and subparts, resulting in $53\times$ more masks than any existing video segmentation dataset.

2.3.2 Architecture and Method

Architectural Overview

The Segment Anything Model 2 (SAM 2) can be seen as a generalization of SAM to the video and image domain, taking point, box, and mask prompts on individual frames to define the spatial extent of the object to be segmented spatio-temporally. Spatially, the model behaves similarly to SAM, utilizing a promptable and lightweight mask decoder that takes an image embedding and prompts to output a segmentation mask for the frame. Prompts can be iteratively added on a frame in order to refine the masks.

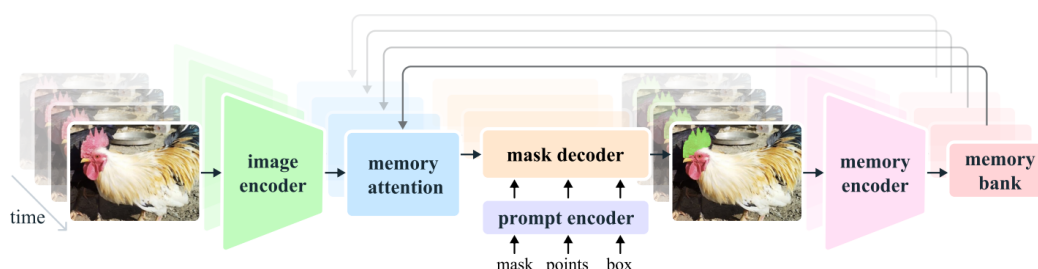


Figure 2: Structural schema of the SAM 2 architecture. Video frames are processed in a streaming fashion by the image encoder. The memory attention module conditions the current frame features on the past frame representations stored in the memory bank. The prompt encoder and mask decoder process sparse or dense interactions alongside the conditioned embeddings to generate masklet predictions and object presence scores, which are subsequently routed through the memory encoder to update the memory bank.

As illustrated in Fig. 2, the frame embedding used by the SAM 2 decoder is not directly from an image encoder and is instead conditioned on memories of past predictions and prompted frames. It is possible for prompted frames to also come from the future relative to the current frame. Memories of frames are created by the memory encoder based on the current prediction and placed in a memory bank for use in subsequent frames. The memory attention operation takes the per-frame embedding from the image encoder and conditions it on the memory bank, before the mask decoder ingests it to form a prediction.

Core Subsystems and Components

- **Image Encoder:** For real-time processing of arbitrarily long videos, a streaming approach is employed, consuming video frames as they become available. The image encoder is only run once for the entire interaction and its role is to provide unconditioned tokens (feature embeddings) representing each frame. SAM 2 uses a Masked Autoencoder (MAE) pre-trained Hiera image encoder, which is hierarchical, allowing the framework to use multiscale features during the decoding process.
- **Memory Attention:** The role of memory attention is to condition the current frame features on the past frame features and predictions as well as on any new prompts. The architecture stacks L transformer blocks, where the first block takes the unconditioned image encoding from the current frame as input. Each block performs self-attention, followed by cross-attention to memories of prompted or unprompted frames and object pointers stored in a memory bank.
- **Prompt Encoder and Mask Decoder:** The prompt encoder is identical to SAM's design and can be prompted by clicks (positive or negative), boxes, or masks to define the extent of the object in a given frame. Sparse prompts are represented by positional encodings summed with learned embeddings for each prompt type, while dense mask prompts are embedded using convolutions and summed with the frame embedding. The decoder stacks "two-way" transformer blocks that update prompt and frame embeddings. For ambiguous prompts, the model predicts multiple masks to ensure valid outputs. An additional head is added to predict whether the object of interest is present on the current frame to handle occlusions.
- **Memory Encoder:** The memory encoder generates a spatial memory by downsampling the output mask using a convolutional module and summing it element-wise with the unconditioned frame embedding from the image encoder. This combined representation is followed by lightweight convolutional layers to fuse the spatial and contextual information.
- **Memory Bank:** The memory bank retains information about past predictions for the target object in the video by maintaining a First-In, First-Out (FIFO) queue of memories of up to N recent frames, and stores information from prompts in a FIFO queue of up to M prompted frames. Both sets of memories are stored as spatial feature maps. In addition to the spatial memory, the module stores a list of object pointers as lightweight vectors for high-level semantic information.

Training Methodology

The model is trained jointly on image and video data. The system simulates interactive prompting of the model by sampling sequences of 8 frames and randomly selecting up to 2 frames to prompt. It probabilistically receives corrective clicks which are sampled using the ground-truth masklet and model predictions during training.

The training task is to sequentially and interactively predict the ground-truth masklet. Initial prompts to the model can be the ground-truth mask with a probability of 0.5, a positive click sampled from the ground-truth mask with a probability of 0.25, or a bounding box input with a probability of 0.25.

2.4 RIFE Model (Real-Time Intermediate Flow Estimation)

2.4.1 Overview and Motivation

Overview

Real-time video frame interpolation (VFI) is highly useful in video processing, media players, and display devices. The Real-Time Intermediate Flow Estimation (RIFE) algorithm introduces a highly efficient, end-to-end deep learning framework designed specifically for flow-based video frame interpolation. Unlike traditional approaches that rely heavily on heavy pre-trained external optical flow models or cumbersome multi-step pipeline combinations (such as separate flow refinement models or flow reversal layers), RIFE utilizes a customized, unified neural network named IFNet.

Given two consecutive input RGB frames alongside a target timestep provided via a temporal encoding input, RIFE directly estimates the intermediate bidirectional optical flows from the perspective of the target frame to be generated. The entire framework operates dynamically via a coarse-to-fine strategy with progressively increasing resolution, enabling arbitrary-timestep frame interpolation with much faster speed and high-quality structural fidelity.

Motivation

The architectural design and training pipeline of RIFE are fundamentally motivated by the following concepts and limitations identified in existing video frame interpolation models:

- **The Inefficiency of Indirect Intermediate Flow Estimation:** Common flow-based VFI methods involve a decoupled two-step framework: first computing bi-directional optical flows from external models, and then reversing and refining them to approximate the intermediate flows. This creates a “chicken-and-egg” problem because the intermediate frame is not available beforehand. As objects shift from frame to frame, hand-engineered flow reversal modules often produce severe flaws and artifacts around motion boundaries. Instead of introducing inaccurate motion modeling, RIFE demonstrates that it is better to make a lightweight CNN learn the intermediate flow estimates directly end-to-end.
- **Eliminating Reliance on Heavy External Components:** Many state-of-the-art frameworks require additional heavy components—such as image depth models, independent flow refinement models, and flow reversal filters—to compensate for the defects of intermediate flow approximation. RIFE aims to maintain the conciseness of a direct estimation pipeline, eliminating any reliance on pre-trained optical flow models that are not explicitly tailored for VFI tasks.
- **Lack of Direct Motion Supervision:** Most existing VFI models are optimized using only the final pixel-wise reconstruction loss. However, propagating gradients of pixel-wise loss across a spatial warping operator is highly inefficient for flow estimation. Lacking explicit supervision designed for motion fields degrades the overall performance of VFI models. RIFE addresses this by introducing an intermediate supervision scheme that directly guides the flow approximation without requiring datasets with ground-truth optical flow labels.

2.4.2 Architecture and Method

Pipeline Overview

The overall pipeline of RIFE is illustrated in Fig. 3. Given a pair of consecutive RGB frames and a target timestep encoded as a separate input channel, the framework directly feeds them into the Intermediate Flow Network (IFNet) to approximate the intermediate bidirectional flows and a pixel-wise soft fusion map.

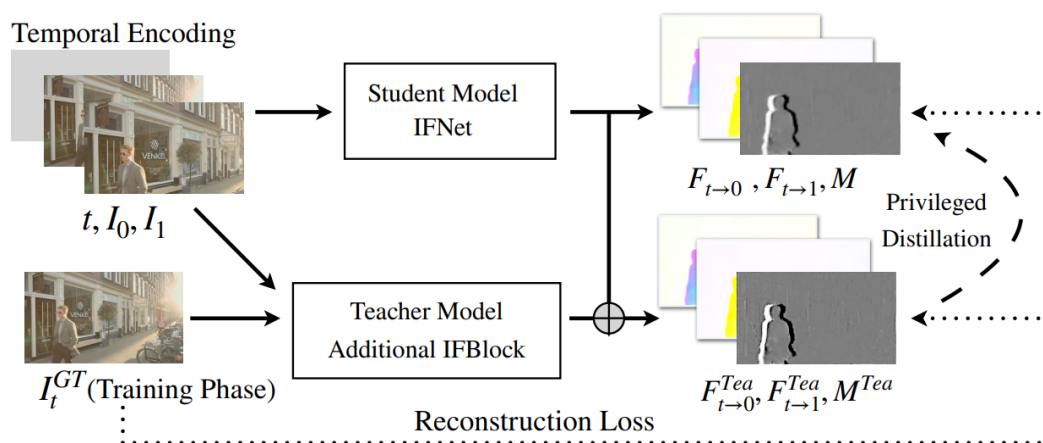


Figure 3: The overall pipeline and training architecture of RIFE. The student network directly estimates bidirectional flows and fusion masks using successive IFBlocks, while the teacher block provides explicit motion supervision via privileged distillation during training.

The initial intermediate reconstructed frame is synthesized by applying a backward warping function on the input frames using the estimated intermediate optical flows, and then combining them linearly based on the weights computed by the soft fusion mask. This approach ensures that areas transitioning due to occlusion or exposed backgrounds are blended smoothly.

To refine high-frequency areas and reduce localized artifacts, an encoder-decoder CNN named RefineNet processes this initial prediction to produce a reconstruction residual. This residual is then added directly to the fused image to generate the final, sharp, high-resolution output frame.

Intermediate Flow Estimation (IFNet)

To handle large motions, IFNet adopts a coarse-to-fine strategy with progressively increasing resolution using a stacked hourglass structure. The intermediate flow fields and the soft fusion mask are iteratively updated via successive convolutional blocks termed IFBlocks. In each block, the system updates the current estimation of the flows and the mask by computing their operational residuals.

RIFE employs a total of 3 IFBlocks with decreasing downsampling scales. To ensure extreme efficiency on resource-constrained devices and embedded NPUs, these blocks completely avoid expensive operators like cost volumes, relying strictly on plain 3×3 convolutions and deconvolutions as their fundamental building blocks.

Privileged Distillation Scheme

Directly approximating intermediate flows is challenging because the target frame is missing during inference. As shown in Fig. 3, during the training phase, RIFE introduces a unique privileged distillation scheme to stabilize IFNet training and improve performance.

An additional teacher block is stacked onto the pipeline during training. This teacher model is granted access to the ground-truth intermediate frame as privileged information, allowing it to refine the student's results and produce highly accurate teacher flows. The student network is explicitly supervised by a linear combination of a Laplacian pyramid reconstruction loss and a privileged distillation loss, which minimizes the structural distance between the student's iterative flow approximations and the teacher's privileged fields.

Crucially, the teacher block is completely discarded after the training phase, ensuring that the student model runs at maximum speed during inference without any extra computational cost.

2.5 SPAN Model (Swift Parameter-free Attention Network)

2.5.1 Overview and Motivation

Overview

Single Image Super-Resolution (SISR) is a foundational task in low-level computer vision aimed at reconstructing high-resolution (HR) images from their low-resolution (LR) counterparts. To maintain real-time throughput and high fidelity within the automated visual processing framework, the Swift Parameter-free Attention Network (SPAN) introduces a highly efficient, lightweight deep learning architecture optimized for edge deployment.

While conventional state-of-the-art super-resolution networks rely heavily on complex, parameterized attention mechanisms or resource-intensive Vision Transformers (ViTs) to capture contextual details, SPAN establishes a novel, non-parametric attention paradigm. By extracting spatial and channel dependencies directly through symmetric activation functions and residual topologies, the model achieves a competitive trade-off between inference speed, parameter footprint, and structural restoration quality. Notably, this architecture demonstrated empirical superiority by winning first place in both the overall performance and runtime tracks of the prestigious NTIRE 2024 Efficient Super-Resolution Challenge.

Motivation

The architectural design of SPAN is motivated by two critical limitations observed in existing efficient super-resolution (ESR) models:

- **Computational Overhead of Conventional Attention:** Although attention mechanisms have significantly improved SISR performance, they typically result in complex network structures and a large number of additional parameters, leading to slow inference speed and large model size. This limits their applicability in efficient, high-speed computing scenarios such as SISR on resource-constrained mobile devices.

- Inefficiency of Parameter- and FLOPs-Reduction Techniques: Many existing ESR models reduce model complexity through group convolution or depth-wise separable convolution. However, simply reducing FLOPs or parameter counts does not always yield meaningful gains in inference speed, and can also decrease model accuracy. Other models reduce parameter size through feature sharing and non-attention branch downsizing, yet still contain many parameters within complex computational structures, resulting in long running times. Maintaining a simple network topology is therefore essential for fast inference.

To address these limitations, SPAN proposes a parameter-free attention mechanism that generates attention maps directly from existing convolutional features via symmetric activation functions, without any additional trainable parameters. This unifies feature extraction and attention mapping under a single shared set of convolutional weights, enhancing high-contribution information and suppressing redundant information while preserving a simple network topology for fast inference.

2.5.2 Architecture and Method

Pipeline Overview

The global processing pipeline of SPAN is illustrated in Fig. 4. Given a single low-resolution input image I_{LR} , an initial 3×3 convolutional layer projects it into a C' -channel feature map O_0 , ensuring that each subsequent SPAB block receives a consistent number of input channels.

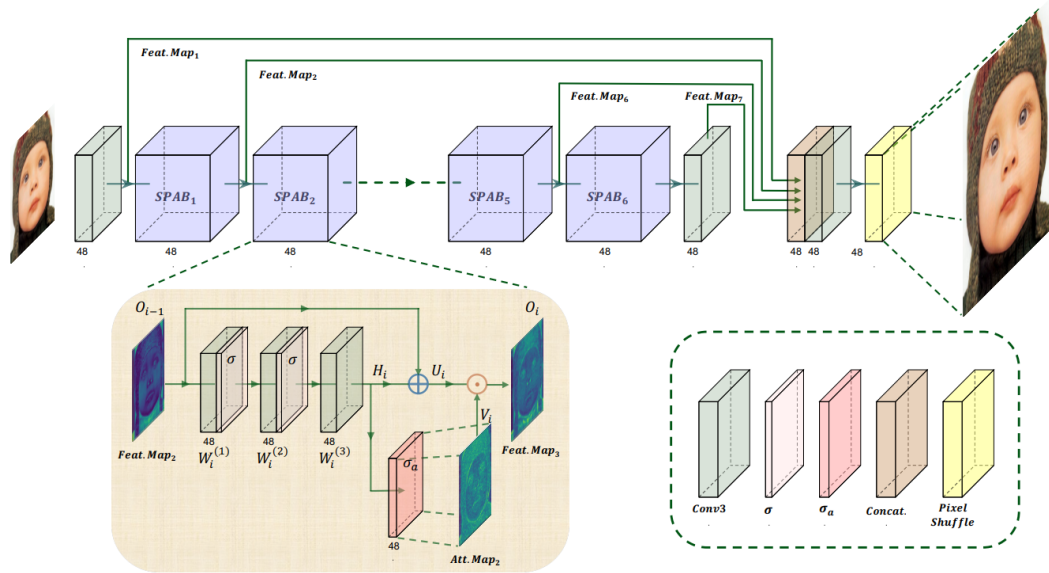


Figure 4: The proposed SPAN architecture. The yellow area indicates the internal structure of each SPAB module. $Att.Map_2$ denotes the generated attention map. Input is a low resolution image, and output is a high resolution image.

The projected feature map O_0 is then sequentially processed through six consecutive Swift Parameter-free Attention Blocks (SPAB), each extracting progressively higher-level features. To capture multi-scale hierarchical information, the outputs of the initial feature map O_0 , the first SPAB block (O_1), the fifth SPAB block (O_5), and a 3×3 convolution of the sixth SPAB block output (O_6) are concatenated into a $4C'$ -channel feature map O .

This aggregated feature map is then passed through a 3×3 convolutional layer to produce an r^2C' -channel feature map, where r is the super-resolution scale factor.

Finally, a `PixelShuffle` upsampling operator rearranges the elements of this feature map to reconstruct a high-resolution output image I_{HR} of spatial dimensions $rH \times rW$.

The SPAB Module

The fundamental computational unit of SPAN is the SPAB module. Within each SPAB, input features are processed through three sequential 3×3 convolutional layers to extract higher-level features H_i . These extracted features are combined with a residual connection from the block’s input to form the pre-attention feature map $U_i = O_{i-1} \oplus H_i$.

The attention map V_i is then obtained by passing H_i directly through an activation function $\sigma_a(\cdot)$ that is symmetric about the origin — requiring no additional trainable parameters. The final output of each SPAB block is produced by element-wise multiplication of the feature map and the attention map:

$$O_i = U_i \odot V_i$$

This design allows the network to enhance high-contribution information and suppress redundant information without increasing model complexity.

Symmetric Activation

A key design requirement in SPAB is that the attention map must be computed without any additional learnable parameters. This places a precise mathematical constraint on the choice of activation function σ_a : it must be symmetric about the origin.

The paper demonstrates that a origin-symmetric (odd) activation function correctly preserves both the sign and magnitude of convolutional outputs when computing the attention map. The sign of a convolutional response encodes directional information, while its magnitude encodes feature strength — both of which must be retained for the attention mechanism to function correctly. Using a non-symmetric function such as `Sigmoid` would map all outputs to positive values, discarding the sign information and causing information loss in the attention field. The SPAN paper theoretically shows that with a symmetric activation and residual connections,

CHAPTER 2. LITERATURE REVIEW

the SPAB block achieves the same role as a conventional parameterized attention mechanism: enhancing high-contribution features while suppressing redundant ones — but with zero additional parameters.

Chapter 3

Methodologies

3.1 Introduction

VISUALine is organized around a pipeline-based architecture. A workflow starts as a YAML configuration file, is executed by the backend runtime, and is exposed to the user through the desktop interface. The same workflow description is therefore used both for processing and for interface generation.

The system contains four main layers:

1. Pipeline layer. YAML files define workflow names, node sequences, node parameters, and optional UI metadata.
2. Execution layer. The PipelineManager and TaskExecutor create node instances, prepare models, and run the workflow on images, batches, or videos.
3. Resource and video I/O layer. The ResourceManager handles model reuse and GPU memory pressure, while the VideoProcessor handles FFmpeg-based video input, output, and audio preservation. PyTorch is used for model execution [1], while FFmpeg is used for video/audio handling [2].
4. Application layer. A FastAPI backend manages jobs and progress, while the Electron/Vue frontend provides the local desktop interface.

The user interacts with the desktop app. After selecting a workflow, the interface reads the workflow metadata and renders the available controls. When processing starts, the backend resolves the selected pipeline, applies any UI overrides, and

executes the nodes in order. Progress is streamed back to the interface, and the final output can be previewed and exported.

This architecture was chosen because VISUALine is not meant to contain one hard-coded pipeline. The system should support different workflows with different models, parameters, and input types while keeping the execution logic consistent.

3.2 Pipeline Design

VISUALine workflows are defined using YAML files. Each file contains the pipeline name, optional UI metadata, and a list of processing nodes. The backend uses the pipeline section to execute the workflow. The frontend uses the UI section to display the workflow name, category, description, input type, and editable controls. A simplified example is shown below:

Listing 3.1: Simplified YAML workflow configuration for fast cinematic prompt blur.

```
pipeline_name: 01_Cinematic_Prompt_Blur_FAST
ui:
  display_name: Cinematic Prompt Blur - Fast
  category: AI Editing
  description: Keep the prompted subject sharp while blurring
    the background.
  input_types: [video, gif, image]
  speed: medium
  supports_prompt: true
  controls:
    - key: prompt
      label: Subject prompt
      type: text
      default: person
  node: 0
  param: prompt
  pipeline:
    - class: segmentation.prompt_box_detection_node.
      PromptBoxDetectionNode
      params:
        prompt: person
        conf: 0.30
    - class: utils.soft_box_mask_node.SoftBoxMaskNode
      params:
        expand_ratio: 0.35
```

```
- class: utils.blur_node.BokehBlurNode
  params:
    blur_intensity: 41
```

This structure keeps the workflow readable. It also avoids tying the interface to a single fixed pipeline. If a new workflow exposes a prompt, confidence threshold, blur strength, output size, or interpolation setting, the control can be added to the YAML file and rendered by the desktop interface. This design is especially useful for an applied system because the workflows are expected to change during development. Models may be replaced, parameters may be tuned, and new nodes may be added. Keeping workflow behaviour in configuration files makes these changes easier to manage and easier to compare.

3.2.1 Node Class Resolution and Instantiation

Each node entry in the pipeline section specifies a fully qualified class path. At load time, the pipeline manager resolves this path using the language's dynamic import mechanism: the string is split at the last dot to separate the module path from the class name, the module is imported at runtime, and the class is extracted and validated against a predefined abstract base interface. This means that new node types can be added to the system by simply placing a source file in the appropriate directory. No registry, factory function, or centralised import list requires modification. The parameters dictionary is passed directly to the node constructor, and each node interprets its own parameter space independently, preserving separation of concerns across the pipeline.

3.2.2 Control-to-Node Binding Mechanism

Each entry in the controls list establishes a binding between a frontend widget and a backend node parameter through two fields. The `node` field references the zero-indexed position of the target node in the pipeline list. The `param` field identifies the specific key within that node's parameters dictionary. When a user modifies a control, the new value is written into the corresponding node's configuration before execution begins. The control schema supports five widget types: `text` for free-form string input, `slider` for continuous numeric ranges with configurable minimum, maximum, and step values, `select` for discrete sets of choices, `toggle` for boolean switches, and `number` for free-form numeric input. Any control may be marked as

advanced to distinguish commonly adjusted parameters from less frequently modified ones.

3.2.3 Multi-Level Configuration Hierarchy

Pipeline-specific YAML files are complemented by a global base configuration that defines system-wide defaults for hardware acceleration policy, default batch size for frame grouping, model cache capacity, logging behaviour, and directory layout for input, output, and temporary files. Pipeline configurations are loaded independently of the base configuration. The two are merged at the application layer where the metadata service supplements pipeline information with system-level defaults before returning them to the frontend.

3.2.4 Workflow Variant Pattern

Several workflow pairs in the library follow a variant pattern where the same user-facing effect is achieved through different node compositions, trading speed for quality. The cinematic blur workflow, for instance, exists in two variants: a fast variant using bounding-box detection with soft masks, and a high-quality variant that substitutes a segmentation and tracking stage for more precise subject boundaries. Similarly, the old-video enhancement workflow offers a fast variant using a single enhancement node and a super-resolution variant that inserts an AI upscaling model between pre-enhancement and post-enhancement stages. Because the pipeline definition is external to the runtime, creating a new variant requires only copying an existing YAML file and modifying the node list. No runtime code changes, no new API endpoints, and no frontend modifications are needed beyond the automatic rendering of any new controls declared in the UI metadata section.

3.3 Workflow Library

The current VISUALine workflow library contains seven user-facing pipelines. They were selected because they cover different parts of the system and produce results that can be inspected visually. Some workflows focus on speed, while others focus on higher output quality.

3.3.1 Cinematic Prompt Blur – Fast

This workflow detects a prompted subject using open-vocabulary detection, converts the detection boxes into a soft mask, and applies blur to the background. It is intended for cases where speed matters more than exact subject boundaries.

Pipeline: `PromptBoxDetectionNode` → `SoftBoxMaskNode` → `BokehBlurNode`

3.3.2 Cinematic Prompt Blur – HQ SAM2

This workflow uses prompt-based detection followed by SAM2-style video tracking. The goal is to produce more accurate masks across frames before applying background blur. It is slower than the fast version, but it can preserve subject boundaries more clearly.

Pipeline: `PromptBoxDetectionNode` → `SAM2TrackingNode` → `BokehBlurNode`

3.3.3 Privacy Redaction

This workflow detects sensitive visual regions, such as people or faces, and applies a redaction effect. It is designed for cases where a user wants to share footage while reducing privacy risks.

Pipeline: `PromptBoxDetectionNode` → `BoxRedactionNode`

3.3.4 Old Video Enhancement – Fast

This workflow applies efficient enhancement operations such as denoising, contrast correction, sharpening, saturation adjustment, and gamma correction. It is intended as a lightweight option for quick visual improvement.

Pipeline: `VideoEnhanceNode`

3.3.5 Old Video Enhancement – SPAN x4

This workflow combines pre-enhancement, AI super-resolution using SPAN, and post-enhancement. It targets low-resolution or degraded videos where stronger up-scaling is needed, but it requires more processing time.

Pipeline: VideoEnhanceNode $\rightarrow$ SPANNode $\rightarrow$ VideoEnhanceNode

3.3.6 Smart Vertical Reframe

This workflow detects a prompted subject and crops the video into a vertical aspect ratio while smoothing the crop movement. It is useful for adapting horizontal videos to short-form formats such as reels or shorts.

Pipeline: PromptBoxDetectionNode $\rightarrow$ SmartReframeNode

3.3.7 Slow Motion RIFE x2

This workflow uses RIFE-style frame interpolation to generate intermediate frames and produce a smoother slow-motion output.

Pipeline: RIFENode

3.4 Dynamic Upscaling Overview

3.4.1 What is Dynamic Upscaling?

Dynamic upscaling is an intelligent video super-resolution approach that selectively processes only the parts of video frames that contain actual motion or change, while bypassing static regions that don't require enhancement.

3.4.2 Problem Solved

Traditional VSR processes every pixel in every frame, causing:

- Prohibitive computational costs on consumer hardware
- Memory bottlenecks leading to OOM errors
- Limited deployment to cloud infrastructure only

3.4.3 Core Insight

Video sequences contain significant spatio-temporal redundancy:

- Static backgrounds remain unchanged across frames
- Moving objects are the only parts requiring enhancement
- Most of the time, only small portions of frames actually change

3.4.4 Dynamic Approach

Instead of processing entire frames, dynamic upscaling:

- Detects motion using simple difference heuristics
- Routes dynamic blocks to the VSR model for processing
- Caches static blocks from previous frames
- Periodically refreshes to prevent artifacts

3.4.5 Benefits

- Memory Efficient: Eliminates OOM errors on consumer GPUs
- Speed Optimized: Up to 33.85× faster processing
- Quality Preserved: Maintains high-fidelity output
- Hardware Friendly: Works on consumer-grade devices
- Practical Deployment: Enables local edge AI without cloud dependency

3.5 Node Interface and Data Flow

Each processing step in VISUALine is implemented as a node. A node performs one main operation, such as detecting objects, generating masks, applying blur, enhancing frames, upscaling images, reframing a video, or interpolating frames. The

system defines a fixed lifecycle contract that every node must follow, ensuring that the runtime can manage model loading, inference, and cleanup uniformly regardless of which specific operation the node performs.

3.5.1 Node Lifecycle Contract

Every node adheres to a three-stage lifecycle enforced by an abstract base interface:

1. `setup(device)` prepares the node and loads any required model. This stage is called once before any data is processed. It receives the target compute device (GPU or CPU) and is responsible for allocating GPU memory, loading pretrained weights from disk, initialising model internals, and configuring inference parameters such as precision mode or tile size. By isolating resource allocation in a dedicated stage, the runtime can sequence model loading across multiple nodes without interference.
2. `process(data)` applies the node operation. This stage receives input data and returns transformed output. It is called repeatedly during execution. The input and output are both expected to conform to the system's shared tensor convention. The node is not responsible for device transfers, batch management, or memory cleanup; those concerns belong to the execution runtime. This separation allows individual nodes to remain stateless and focused purely on their transformation logic.
3. `teardown()` releases model references or temporary resources when processing is complete. This stage is called once after all data has been processed. It clears GPU memory, removes model references, and resets internal state so that the node can be garbage-collected or reloaded cleanly. Without explicit teardown, long-running applications on consumer GPUs would accumulate memory across pipeline reloads.

This three-stage design decouples resource management from computation. Model loading occurs once per pipeline execution rather than once per frame. Inference is batched and sequenced by the runtime without each node needing to manage device context. Cleanup is deterministic rather than relying on Python's garbage collector, which is especially important on GPU memory where delayed deallocation can cause out-of-memory failures.

3.5.2 Shared Data Convention

The data passed between nodes can be an image tensor, a video tensor, or a dictionary that contains tensors alongside detection boxes, masks, or metadata. To make utility nodes interoperable without case-by-case conversion logic, the system defines a fixed tensor shape convention for all image-like data:

- RGB input: $(B, 3, H, W)$. A batch of B colour images, each with three channels (red, green, blue), height H , and width W . Values are normalised floats in the range $[0, 255]$ before processing and are clamped to this range on output.
- RGB+Mask input: $(B, 4, H, W)$. A batch of B images where the fourth channel represents a mask. The mask can denote the subject region to preserve or the region to modify, depending on the node's semantics. Utility nodes normalise the mask internally so that outputs from different models can be used together regardless of whether the source produces binary, soft, or probability-valued masks.
- Temporal RGB input: $(B, T, 3, H, W)$. A batch of B temporal windows, each containing T consecutive RGB frames. This shape is used by nodes that require motion context, such as tracking or optical flow models. The temporal dimension T may vary across different pipeline configurations.
- Temporal RGB+Mask input: $(B, T, 4, H, W)$. A batch of B temporal windows with T frames, each carrying an additional mask channel. This shape supports video segmentation and tracking pipelines where the mask must propagate across time.

All four tensor shapes share a common design principle: the batch dimension B always comes first, the channel dimension always comes second (or third for temporal), and the spatial dimensions are always last. This layout is chosen to match PyTorch's native tensor format, which avoids unnecessary transposition overhead when moving data between the node interface and the underlying model implementations.

3.5.3 Mask Channel Semantics

When a fourth channel is present, it represents a mask. A value of 1 (after normalisation) represents the selected subject or protected region, depending on the node. In blur nodes, the mask indicates the subject to keep sharp while the background is blurred. In redaction nodes, the mask indicates the region to obscure. In

background removal nodes, the mask indicates the foreground to retain. This interpretation is determined by each node's implementation, but the convention ensures that the mask channel always occupies the same position in the tensor, so utility nodes and visualisation tools can operate generically on any pipeline's output.

The mask format itself is flexible. Some upstream models produce hard binary masks (every pixel is either 0 or 1), while others produce soft probability maps or boundary-aware distance fields. Each node is responsible for normalising incoming masks to its expected range before applying its operation. A blur node, for instance, may treat a soft mask as a blending weight for the blur effect, producing a smooth transition between the blurred background and the sharp subject. This flexibility would be difficult to achieve without a consistent channel-position convention, because each pair of nodes would otherwise need custom integration logic.

3.5.4 Rationale for a Fixed Tensor Convention

Without a shared data convention, each node pair would need to negotiate its own data format. A detection node might output boxes in pixel coordinates while a masking node expects normalised coordinates. A segmentation node might output a single-channel mask while a compositing node expects a four-channel RGBA tensor. These mismatches accumulate across multi-node pipelines and produce bugs that are difficult to trace because the incorrect values may propagate through several nodes before producing visible artefacts.

The fixed tensor convention eliminates this class of errors at the architectural level. Every node knows that its input will be one of the four standard shapes and that the mask channel, if present, will occupy the same position in every tensor. Nodes that need additional metadata, such as bounding-box coordinates or detection confidence scores, can receive them through an optional dictionary wrapper while keeping the tensor data itself in the standard format. This hybrid approach, with structured tensor shapes for pixel data and flexible dictionaries for metadata, balances the strictness required for interoperability with the expressiveness required for complex detection and tracking pipelines.

3.6 Execution Runtime

The execution runtime is responsible for running the configured workflows efficiently on local hardware. Since the system targets consumer GPUs, the runtime has to

avoid unnecessary model loading, reduce memory pressure, and keep video processing predictable.

3.6.1 PipelineManager

The PipelineManager reads a YAML configuration, creates the required node instances, calls `setup()` on each node, and runs the nodes in order. It supports single images, image batches, and videos. For video workflows, it also tracks properties that affect the output, such as temporal windows and FPS multipliers.

This is needed for pipelines such as RIFE interpolation, where the number of output frames changes. It is also useful for workflows that operate on temporal context instead of processing each frame independently.

The PipelineManager serves as the central orchestrator for all pipeline execution. Its primary responsibility is input routing: it inspects the input path suffix to determine whether the input is a single image, a directory of images, or a video file, and dispatches to the appropriate internal execution path. Each path follows the same high-level structure, but the video path introduces additional complexity because frames must be streamed, batched, temporally windowed, and reassembled into a playable output.

For single-image inputs, the pipeline reads the image using OpenCV, converts from BGR to RGB order, expands the frame into a batch of size one, executes the pipeline, converts the output back to BGR, and writes the result. For image directories, the manager groups images by resolution to avoid shape mismatches within a batch, processes them in resolution-homogeneous groups, and writes each output file with its original filename. This grouping strategy maximises batch utilisation without requiring all input images to share the same dimensions.

For video inputs, the PipelineManager implements a producer-consumer architecture. An FFmpeg reader thread decodes raw RGB24 frames from the input file and places them into a bounded queue. The main thread pulls batches from this queue, runs them through the pipeline, and places the processed output into a second bounded queue. An FFmpeg writer thread pulls from the output queue and encodes the frames into the output file. The bounded queues act as flow regulators: if the GPU processes faster than the encoder can write, the output queue fills and the main thread blocks, preventing unbounded memory growth.

The manager also handles batch-size management. Before processing begins, it probes the output dimensions by running a single sample frame through the

pipeline, which allows it to configure the FFmpeg writer with the correct resolution and channel count before any real frames are processed. During execution, if a CUDA out-of-memory error occurs, the manager halves the batch size recursively and concatenates the partial results. This recovery mechanism allows pipelines to degrade gracefully on hardware with limited VRAM rather than failing entirely.

3.6.2 TaskExecuter

The TaskExecuter prepares the list of `node process()` calls into an execution plan. During inference, it runs the plan sequentially under PyTorch inference mode. This keeps the main processing loop simple and avoids training-related overhead.

The TaskExecuter also helps keep tensor conversion behaviour centralised. Some nodes operate on PyTorch tensors, while others may expect NumPy-style frame data. A consistent execution path reduces the chance of repeated conversion code being scattered across the system.

The TaskExecuter is intentionally lightweight. It does not perform graph optimisation, automatic mixed-precision casting, or device-to-device transfers. Each node is responsible for its own inference behaviour, and the PipelineManager is responsible for all input and output conversions. The TaskExecuter's sole responsibility is to iterate over the compiled node list and call each node's `process()` method in sequence, passing the output of one node as the input to the next.

By wrapping the entire execution loop in PyTorch's inference mode, the TaskExecuter disables gradient computation, autograd tracking, and other training-related machinery that is unnecessary during pipeline execution. This reduces memory consumption and improves inference speed, particularly for models that would otherwise retain intermediate activations for backpropagation. The inference mode context manager is applied at the execution-loop level rather than inside individual nodes, which ensures that no node can accidentally enable gradient tracking regardless of its internal implementation.

The execution plan also supports nodes that produce dictionary outputs containing both tensor data and metadata. A detection node, for instance, may return a dictionary with a tensor key for the image data and a `boxes` key for bounding-box coordinates. The TaskExecuter passes the full dictionary to the next node, which can extract whichever fields it needs. At the end of the pipeline, the manager unwraps the final dictionary by extracting the tensor field, discarding any remaining metadata that is not needed for the output file.

3.6.3 ResourceManager

Some models are expensive to load and may occupy a large amount of GPU memory. VISUALine uses a ResourceManager to cache model wrappers, move models to the selected device, and coordinate cleanup when resources are no longer needed. This is useful when the same workflow is run multiple times or when the user switches between workflows that share components.

Centralising model lifecycle management also makes the system easier to reason about. Instead of each node independently deciding when to load or release a model, the runtime can follow a more controlled resource strategy.

The ResourceManager implements a least-recently-used cache with GPU memory awareness. When a node requests a model, the manager checks whether that model is already loaded in the cache. If it is, the model is promoted to the most-recently-used position and returned immediately. If it is not, the manager checks whether loading the new model would exceed the configured VRAM utilisation threshold. If the threshold would be exceeded, the manager evicts the least-recently-used model from the cache, frees its GPU memory, and repeats until sufficient memory is available or the cache is empty. If a CUDA out-of-memory error still occurs during model loading despite eviction, the manager flushes the entire cache and retries.

This strategy addresses a specific constraint of consumer GPU hardware. Unlike data-centre GPUs with 40 GB or more of VRAM, consumer GPUs typically offer between 8 GB and 16 GB. Loading multiple vision models simultaneously, particularly segmentation and super-resolution models that each require several gigabytes, can quickly exhaust this budget. The LRU eviction policy ensures that the most recently used models remain resident while infrequently used models are transparently unloaded and reloaded on demand.

The ResourceManager also implements a locking mechanism. During pipeline execution, the manager can be locked to prevent new models from being loaded. This is enforced at runtime: if a node attempts to load a model through the manager while a lock is held, the request is queued until execution completes. This prevents a running pipeline from being disrupted by model loading triggered as a side effect of another concurrent operation.

3.6.4 VideoProcessor

Video processing is handled through FFmpeg and FFprobe. The VideoProcessor reads metadata such as frame rate and resolution, manages frame extraction or

streaming where needed, preserves the original audio, and writes the processed output. When available, hardware encoding such as NVENC can reduce export time.

This component is important because video output quality and usability depend not only on the AI model output, but also on the final container, codec, frame rate, and audio handling. A visually correct result is less useful if the exported video cannot be previewed or if the original audio is lost.

The VideoProcessor abstracts all FFmpeg interaction behind three operations: reading, writing, and audio muxing.

The reader operation spawns an FFmpeg subprocess that decodes the input video and writes raw RGB24 pixel data to stdout, which the processor reads as a pipe. Before spawning, it probes the input file with FFprobe to extract metadata including resolution, frame rate, total frame count, and codec information. If the input video exceeds a configured maximum processing resolution, the reader automatically downscales while keeping dimensions even, which prevents shape mismatches in downstream tensor operations that require even spatial dimensions.

The writer operation spawns an FFmpeg subprocess that reads raw pixel data from stdin and encodes it into the output file. The encoder is selected automatically based on the output characteristics. If the processed frames contain an alpha channel (four channels instead of three), the writer uses ProRes 4444 with a mov container, which is one of the few widely supported formats that preserves transparency. For large output resolutions, the writer prefers HEVC encoding, attempting hardware-accelerated encoders first (NVIDIA NVENC, then VideoToolbox on macOS, then AMD AMF or Intel QSV) and falling back to software libx265. For standard resolutions, it follows the same priority chain with H.264 encoders. If the selected encoder is incompatible with the output file extension, the writer corrects the extension automatically to avoid producing unplayable files.

Audio preservation is handled as a separate step after video encoding completes. The processor uses FFmpeg to extract the audio stream from the original input file and mux it into the processed output file without re-encoding. This approach separates the audio handling concern from the GPU processing pipeline, which operates exclusively on visual data. It also avoids the complexity of maintaining audio synchronisation during frame-level processing, since the final muxing step aligns the audio track with the processed video based on their respective timestamps.

3.7 User-Facing Application

VISUALine includes a local desktop interface built with Electron and Vue. The frontend communicates with a FastAPI backend running on localhost. The interface follows a simple workflow: select a pipeline, choose an input file, adjust the shown controls, start processing, monitor progress, and preview the result.

The interface is designed to hide most implementation details. For example, the user does not need to know whether the selected workflow uses box masks, SAM2 masks, frame interpolation, or SPAN super-resolution internally. The UI only exposes the controls that are meaningful for the selected workflow.

The backend provides routes for pipeline discovery, system health checks, media processing, job tracking, and output retrieval. WebSockets are used for progress updates so that the frontend can show the status of a running job without repeatedly refreshing the page.

Local preview support is also part of the application design. The user should be able to compare the input and output inside the same interface, especially for visual tasks such as background blur, redaction, enhancement, and reframing. This makes the system feel closer to a practical editing tool rather than a command-line model demo.

3.7.1 Backend API Architecture

The backend is organised around a set of RESTful endpoints that expose the system's capabilities without requiring direct access to the Python runtime. Each endpoint group maps to a distinct functional area: pipeline discovery for listing available workflows and retrieving their metadata with control schemas, media processing for submitting images and videos to a selected pipeline, job tracking for monitoring active and completed tasks, system health for querying GPU availability and resource utilisation, and output retrieval for downloading or previewing processed results.

Media processing requests are handled asynchronously. When a user submits a file for processing, the backend creates a job record with a unique identifier and returns it immediately, allowing the frontend to begin polling or subscribing to progress updates without blocking. The actual pipeline execution runs on a background thread, freeing the API server to handle other requests while processing is underway. This asynchronous design is essential for video workflows, which may take several minutes to complete.

Progress updates are streamed to the frontend through WebSocket connections. The backend emits events at configurable intervals containing the current frame count, total frame count, elapsed time, and a status string. The frontend uses these events to update a progress bar, estimated time remaining, and status indicator without requiring repeated HTTP polling. When processing completes, the backend sends a final event containing the output file path and duration, after which the frontend enables preview and download controls.

3.7.2 Dynamic Interface Rendering

The frontend never hard-codes the presence of a specific slider, toggle, or text field. Instead, it fetches the pipeline metadata from the backend at startup and renders the interface dynamically based on the `controls` array declared in each workflow's YAML configuration. Each control specifies a widget type (`text`, `slider`, `select`, `toggle`, or `number`), a human-readable label, a default value, and constraints such as minimum and maximum ranges or available options. The frontend maps each type to a corresponding Vue component and renders it in the parameter panel.

Controls marked as `advanced` are hidden behind an expandable section, keeping the default interface focused on the parameters that most users need to adjust. When the user modifies a control, the frontend updates an internal parameters object without submitting anything to the backend. Only when the user clicks the start button are the current parameter values serialised and sent alongside the file upload. This design avoids unnecessary API calls during parameter adjustment and ensures that the backend always receives a consistent set of overrides, regardless of which controls were touched.

3.7.3 Workflow Selection and Metadata Display

The workflow picker is populated from the backend's pipeline discovery endpoint, which scans the configuration directory and returns a list of `PipelineSummary` objects. Each summary includes a display name, category, description, supported input types, expected speed, and tags. The frontend groups workflows by category, displays them as a grid of cards, and allows filtering by input type, speed, or keyword tags.

When the user selects a workflow, the frontend fetches the full detail view from the backend, which includes the complete control schema and node list. The detail

view replaces the workflow picker with the parameter panel. The user can return to the picker at any time to switch workflows, which triggers a new detail fetch and a fresh set of controls. This navigation model treats workflows as interchangeable configurations rather than separate pages, reinforcing the design principle that the interface is driven entirely by metadata.

3.7.4 Local Preview and Output Management

After processing completes, the frontend fetches the output file from the backend and displays it alongside the original input in a side-by-side or split-view comparison panel. The preview component is resolution-aware: it scales large outputs to fit the viewport while preserving aspect ratio, and it provides zoom and pan controls for inspecting fine details. For video outputs, the preview includes a synchronized playback controller that allows the user to scrub through both the input and output simultaneously, making it straightforward to evaluate the effect at any point in the timeline.

The backend retains completed job outputs until the user explicitly clears them or starts a new job. This allows the user to run multiple workflows on the same input, compare the results, and decide which output to export. Export triggers a direct file download through the backend, which serves the file with the correct MIME type and filename.

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter explains the System Analysis and Design of VISUALine. The goal of this chapter is to present a clear technical foundation before implementation, showing how system requirements are translated into a practical and scalable design.

It begins by defining the system scope, major components, and expected behavior from both user and developer perspectives. It then specifies the software and hardware environment, including supported platforms, dependencies, performance considerations, and compatibility constraints needed to run the platform efficiently. In addition, the chapter highlights the main assumptions and external dependencies that may affect deployment, performance, or future extensions.

To describe the system architecture in a structured way, this chapter includes block-level representation and UML-based modeling. The UML diagrams are used to capture different views of the system, such as user interactions, component structure, process flow, and runtime behavior between modules. This multi-view representation helps ensure consistency between requirements, architecture, and implementation decisions.

Finally, the chapter outlines the selected software lifecycle and time plan, including major phases and milestones. Together, these sections provide a complete design reference that supports development, testing, and evaluation in the following chapters.

4.2 System Software and Hardware Requirements

This section defines the software and hardware requirements needed to develop, run, and evaluate the VISUALine system. The requirements are organized to ensure reproducibility, stable performance, and compatibility with AI-based image and video processing workflows.

4.2.1 Software Requirements

The VISUALine platform depends on a Python-based AI stack with supporting tools for model inference, media handling, and user interaction.

- Operating System: Windows 10/11 (WSL2 Linux environment recommended for compiling CUDA extensions), Linux (Ubuntu 20.04+), or macOS (for Apple Silicon GPU/MPS-enabled execution).
- Programming Language: Python 3.11.x (strictly required due to compiler compatibility with GroundingDINO and SAM2 libraries).
- Core Frameworks and Libraries:
 - PyTorch $\geq 2.7.0$ (with CUDA 12.8 support for GPU acceleration or MPS support for macOS),
 - OpenCV headless ≥ 4.10 (image and frame handling),
 - NumPy ≥ 1.26 (tensor and array manipulations),
 - FFmpeg and FFprobe (system binary paths used for audio muxing, video decoding, and hardware-accelerated NVENC encoding),
 - Model-specific dependencies (GroundingDINO compiled from source, SAM-2 tracking library, RIFE, SPAN super-resolution).
- Development Tools:
 - Git (version control),
 - Conda or virtualenv (environment management),
 - IDE/editor (e.g., Cursor, VS Code, or PyCharm).
- Application Interface Layers:
 - Primary Desktop UI: An Electron-Vue frontend coupled with a FastAPI local backend server.

4.2.2 Hardware Requirements

VISUALine supports CPU-only execution, but GPU acceleration (CUDA on Windows/Linux or MPS on Apple Silicon) is strongly recommended for practical processing speed, especially for heavy pipelines.

4.2.2.1 Minimum Requirements

- CPU: Quad-core processor (Intel i5 / Ryzen 5 class or equivalent),
- RAM: 8 GB,
- Storage: 15 GB free space (necessary for PyTorch installation, model weights, and media cache),
- GPU: Optional (CPU mode supported, but results in significantly slower inference).

4.2.2.2 Recommended Requirements

- CPU: 6-core or higher modern processor,
- RAM: 16–32 GB,
- Storage: SSD with at least 30 GB free space,
- GPU: NVIDIA GPU with CUDA support and at least 8–12 GB VRAM (higher VRAM is required for high-resolution video tracking, SAM-2 spatial-temporal tracking, and tiled SPAN super-resolution).

4.2.3 Performance-Related Notes

- Heavy restoration and super-resolution pipelines benefit significantly from GPU execution (CUDA or MPS).
- For large-resolution inputs, tiled inference is implemented to avoid out-of-memory errors on consumer GPUs.
- Processing time depends on input resolution, video length, selected pipeline, and enabled model options (e.g., FP16, tiling, temporal processing).

4.2.4 Compatibility Constraints

- CUDA-enabled features require compatible NVIDIA drivers, CUDA runtime, and C++ compiler tools (MSVC or GCC) to compile custom nodes.
- FFmpeg and FFprobe must be installed and accessible in the system path (PATH) for full video I/O and audio muxing support.
- Model checkpoint files must be available in the configured `weights/` or `exports/` directory.

4.3 Assumptions and Dependencies

The successful deployment, initialization, and execution of the VISUALine system rely on several critical external assumptions and software dependencies:

- **Pre-trained Model Checkpoints (Weights):** The system assumes that the required pre-trained weights (checkpoints) for GroundingDINO, SAM2, SPAN, and RIFE have been downloaded and placed in the configured `weights/` or `exports/` directory within the project root. The dynamic pipeline manager does not perform automatic model downloading during runtime and will raise a `FileNotFoundError` if these files are missing during the node setup phase.
- **C++ and CUDA Build Tools (Compilation Dependency):** Several core models—specifically GroundingDINO and SAM2—require the compilation of custom CUDA/C++ extensions during environment setup (using `vendor/GroundingDINO` as an editable package). It is assumed that the host system has an appropriate C++ compiler (such as MSVC Build Tools on Windows or GCC/G++ on Linux) and the CUDA Toolkit version matching the installed PyTorch binary.
- **System-Level Media Utilities (FFmpeg & FFprobe):** The system's `VideoProcessor` relies on external system calls to FFmpeg and FFprobe via Python subprocesses. It is assumed that these binaries are installed globally on the operating system and registered in the system's environment variable path (PATH). The system does not package these binaries locally.
- **Network Connectivity during Setup vs. Offline Runtime:** An active internet connection is assumed during the initial environment installation (to fetch

Git dependencies, such as the BasicSR and SAM2 repositories, and download weights). However, once the environment is initialized, it is assumed that the system runs fully offline locally to ensure privacy and eliminate latency from cloud services.

- **Input Media Integrity:** It is assumed that input files are standard, non-corrupted image or video formats (e.g., MP4, MOV, AVI, PNG, JPG) supported by FFmpeg, and that they do not possess Digital Rights Management (DRM) restrictions that would prevent decoding.

4.4 Diagrams

This section details the system architecture and execution flow of the VISUALine platform through static layered blocks and dynamic sequence diagrams.

4.4.1 System Architecture Diagram

VISUALine is structured as a 7-layer application running entirely on the host machine. Figure 5 illustrates the layout and data flow.

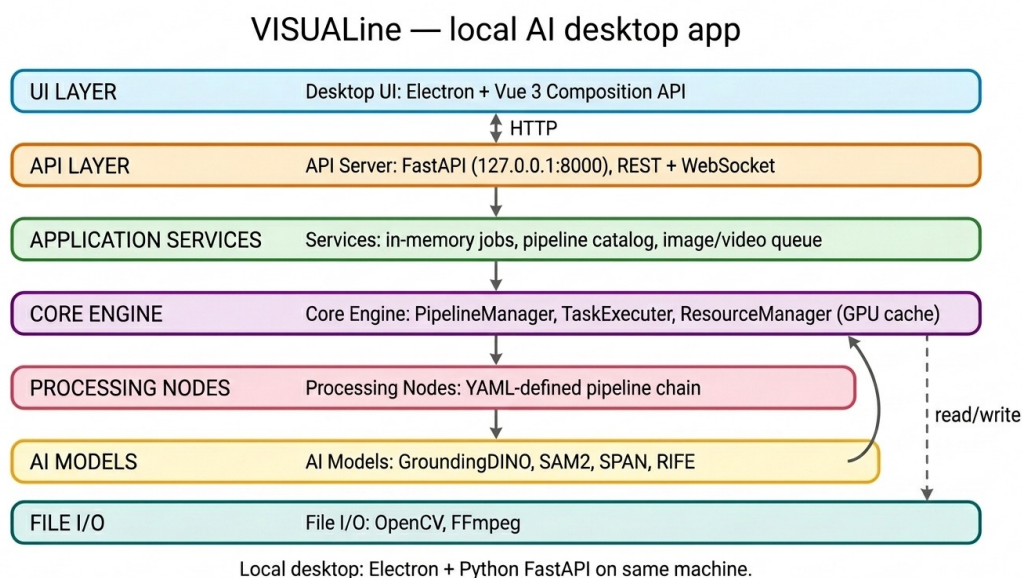


Figure 5: The 7-layer system architecture of VISUALine.

The functions of the seven layers shown in Figure 5 are summarized below:

- UI Layer: Electron + Vue 3 desktop interface for controls and previews.
- API Layer: Local FastAPI server handling REST and WebSocket communications.
- Application Services: Pipeline catalog, task queues, and job management.
- Core Engine: Manages execution graph compilation, execution, and VRAM-aware model caching.
- Processing Nodes: Reusable pipeline components inheriting from a base class (NodeBase).
- AI Models: Deep learning wrappers for GroundingDINO, SAM2, SPAN, and RIFE.
- File I/O: Frame decoding/encoding and audio muxing via OpenCV and FFmpeg.

4.4.2 Pipeline Sequence Diagrams

This section traces the end-to-end execution flow of a media processing task within the system. The diagrams trace the workflow of the Cinematic Prompt Blur (HQ SAM2) pipeline, illustrating how the client interface, FastAPI routes, and GPU execution modules cooperate to process frames.

4.4.2.1 API and UI Initiation Sequence

The execution starts with the client-server interaction. The UI queries configuration schemas, renders controls, submits jobs, and receives progress updates via WebSockets, as shown in Figure 6.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

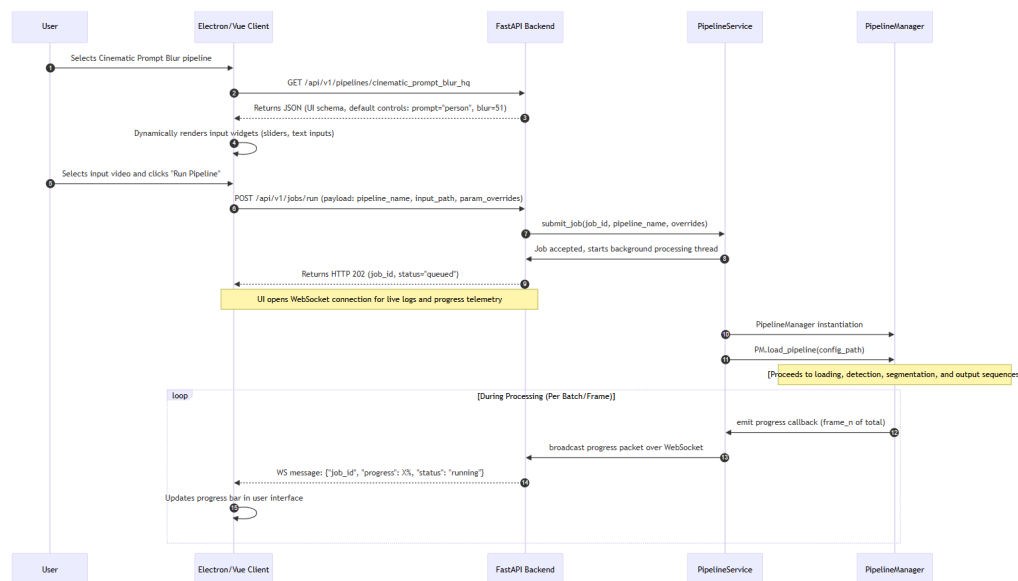


Figure 6: Sequence diagram illustrating API startup, parameters rendering, job queuing, and WebSocket progress updates.

Key operations in this sequence include:

- **Dynamic Configuration:** The UI queries the pipeline schema at startup, allowing widgets (sliders, text inputs) to render dynamically based on the YAML description without hard-coded frontend values.
- **Asynchronous Task Execution:** Job submissions return an immediate HTTP 202 Accepted status, routing the processing task to a background thread to prevent UI lockup.
- **WebSocket Progress Telemetry:** Live frame counts and status logs are pushed back to the client UI over a WebSocket connection for real-time progress bar updates.

4.4.2.2 Pipeline Loading Sequence

Once a job is active, the `PipelineManager` resolves the YAML config, instantiates node objects, caches weight files, and compiles the execution plan, as shown in Figure 7.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

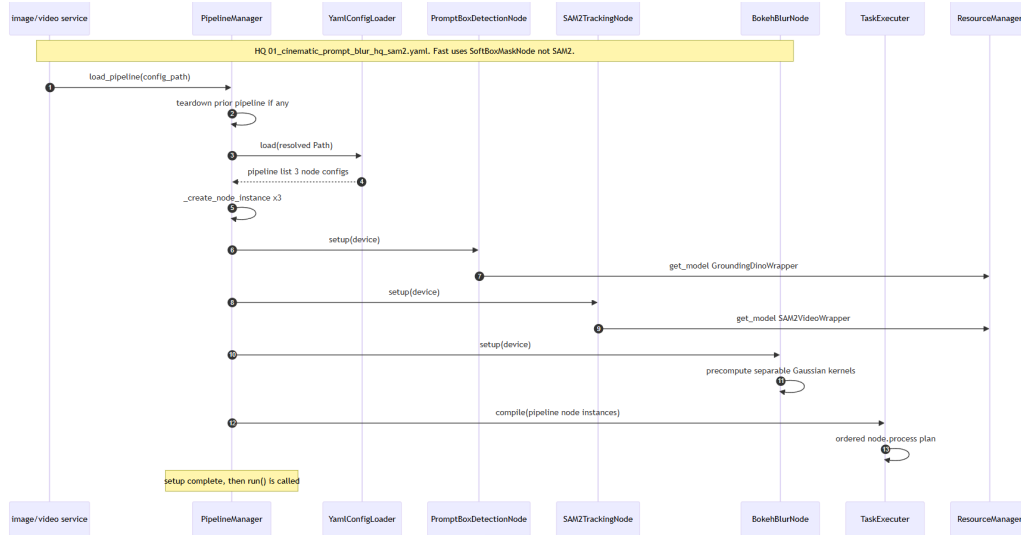


Figure 7: Sequence diagram showing configuration loading, dynamic node setup, and resource allocation.

Key operations in this sequence include:

- **Dynamic Instantiation:** Node classes are dynamically resolved and instantiated from string namespaces declared in the YAML config.
- **Model Cache Query:** Setup methods fetch weight wrappers via the singleton `ResourceManager`, which checks VRAM soft limits and evicts least-recently-used models under memory pressure.
- **Execution Plan Compilation:** The `TaskExecutor` compiles the loaded nodes into an ordered list of callable functions under a unified `torch.inference_mode` decorator.

4.4.2.3 Target Detection Sequence

Input frames are scaled and converted to RGB tensors. Bounding boxes are generated using GroundingDINO based on text prompts, then filtered and cached to optimize speed, as shown in Figure 12.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

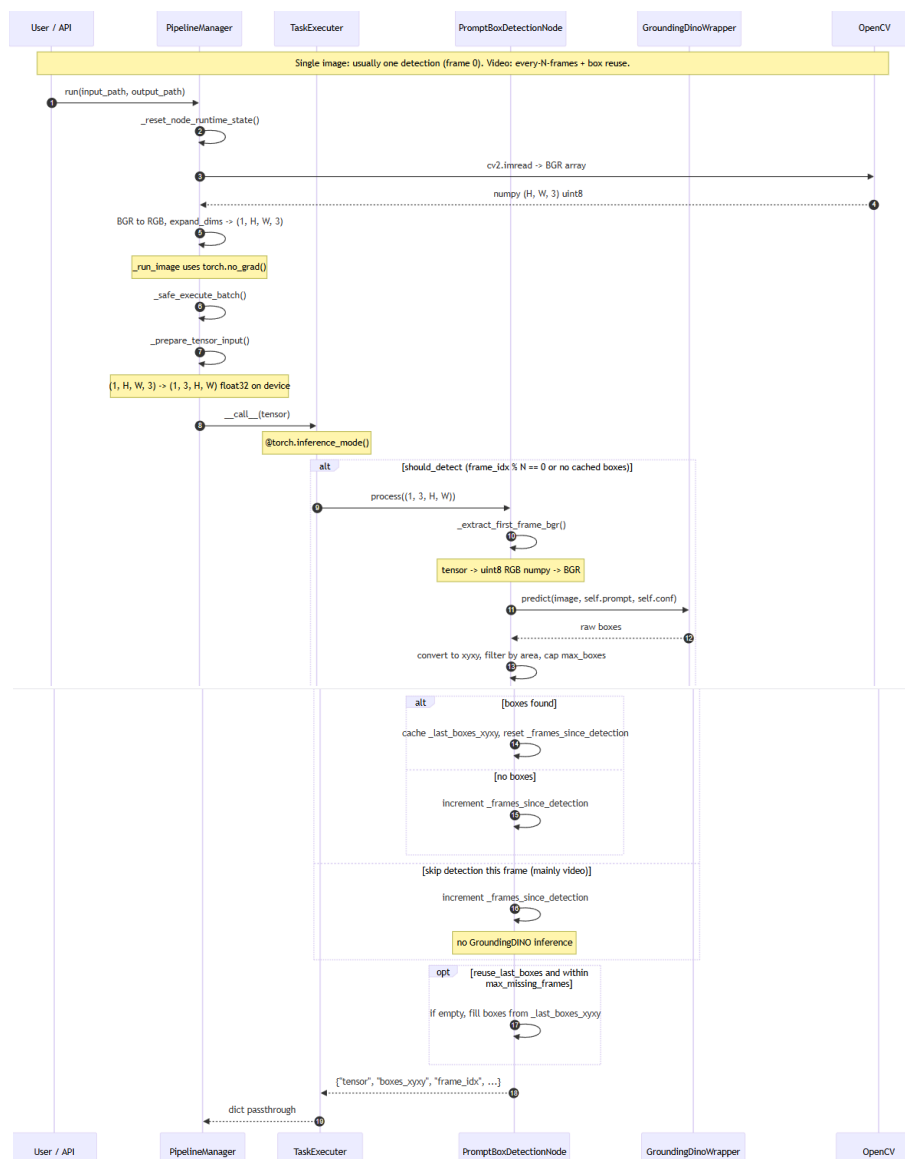


Figure 8: Sequence diagram detailing prompt-based bounding-box detection, filtering, and caching optimizations (Part 1: Data ingestion and pipeline triggers; Part 2: Bounding box inference and caching).

Key operations in this sequence include:

- **Asynchronous Frame Piping:** A dedicated reader thread streams video frames from a subprocessed Ffmpeg pipe into a thread-safe read queue, maintaining a temporal frame window.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

- Temporal Detection Skipping: Bounding box detection is run conditionally (every N frames) rather than on every frame, drastically reducing inference latency.
- Box Cache Reuse: On skipped frames, the node reuses cached bounding boxes from prior detections within a parameterized frame tolerance window.

4.4.2.4 Instance Segmentation and Tracking Sequence

Bounding boxes prompt the SAM2 tracking model, which dumps temporary frames, propagates instance masks forward/backward across time steps, and outputs RGB+Mask tensors, as shown in Figure 9.

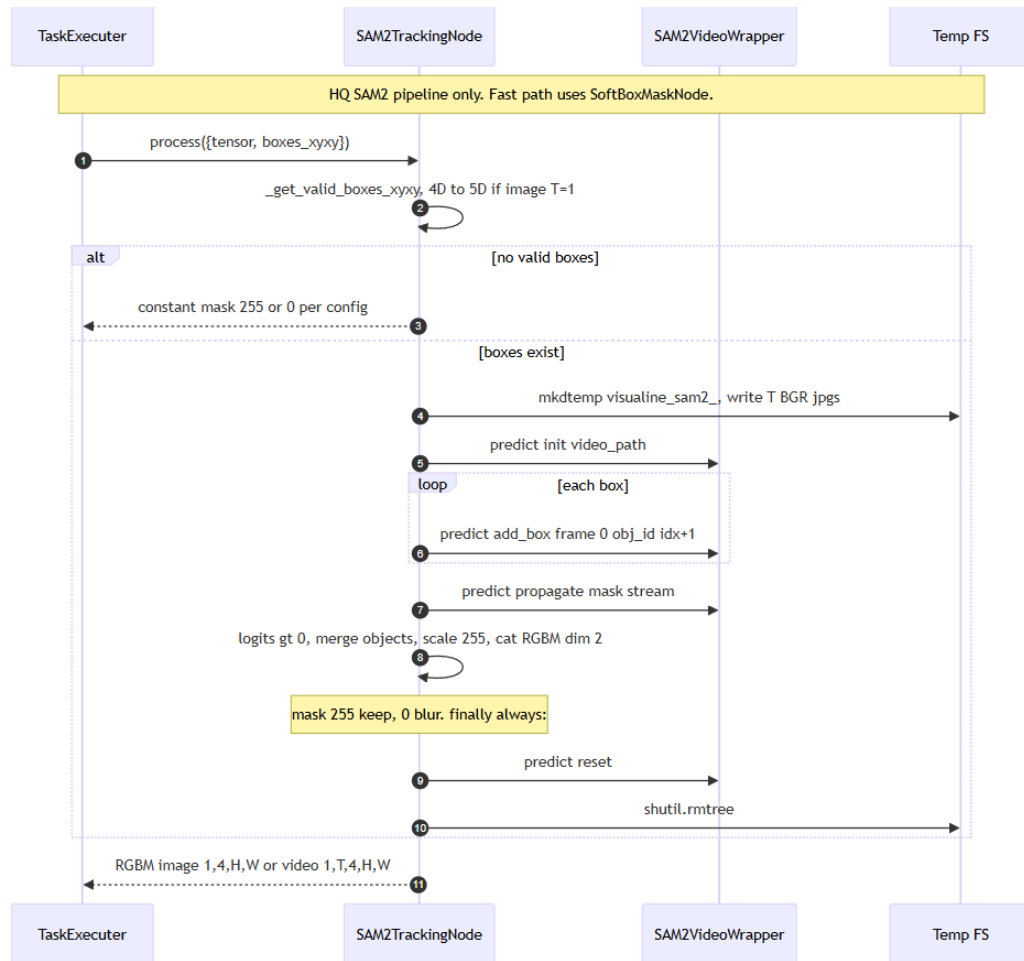


Figure 9: Sequence diagram for SAM2 temporal frame propagation, mask generation, and file cleanup.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

Key operations in this sequence include:

- **Temporal Workspace Isolation:** The node dumps video segment frames to an isolated temporary directory to initialize the SAM2 tracking context.
- **Temporal Mask Propagation:** The SAM2 engine propagates instance tracking forward and backward through the frame sequence using the initial frame prompts.
- **Memory Cleanup:** All intermediate directory structures and wrapper context objects are explicitly destroyed upon completion to prevent VRAM and storage leaks.

4.4.2.5 Blending and Video Output Sequence

The BokehBlurNode or relevant processing nodes blend sharp subjects with modified background channels. Frames are decoded to numpy arrays and written to FFmpeg, with automatic batch division for GPU OOM recovery, as shown in Figure 10.

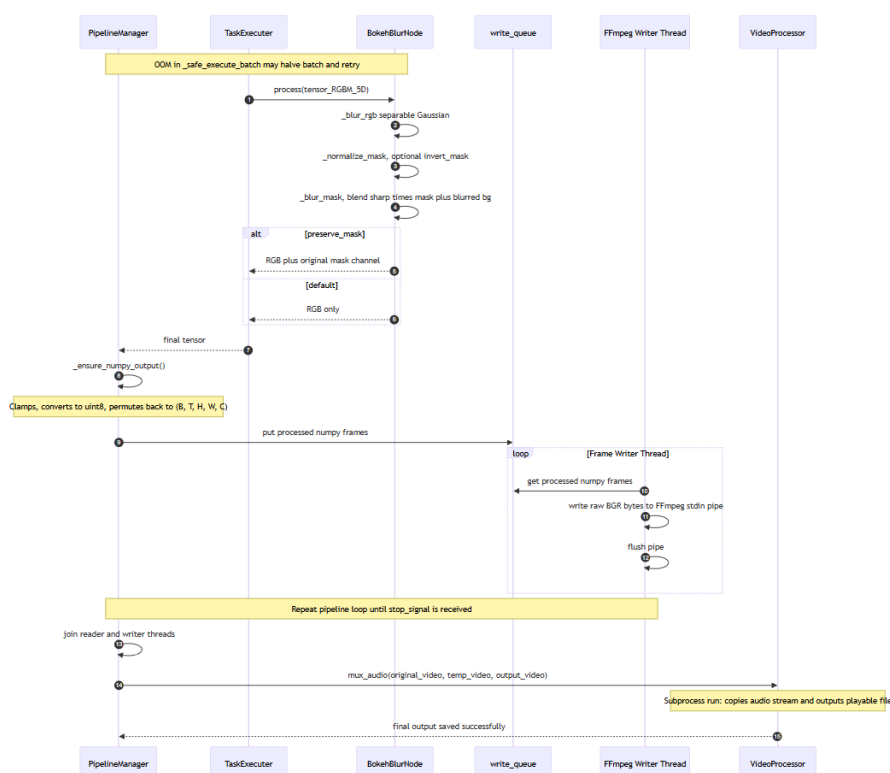


Figure 10: Sequence diagram showing image/video blending, frame output serialization, and OOM recovery handling.

Key operations in this sequence include:

- **Gaussian Blending:** The node applies separable Gaussian filters to blur the background, blending it with sharp foreground regions via a normalized mask tensor.
- **OOM Recovery Engine:** If a PyTorch Out-of-Memory (OOM) exception occurs during execution, the manager catches it, empties the cache, halves the batch size, and recursively retries the execution.
- **FFmpeg Writing Pipe:** A dedicated writer thread consumes processed numpy frames from the write queue and streams BGR bytes to the FFmpeg writer subprocess stdin.
- **Audio Muxing Pass:** Upon pipeline completion, the `VideoProcessor` executes an asynchronous FFmpeg command to copy the audio stream from the original source into the newly rendered video container.

4.4.3 Privacy Redaction Pipeline Sequence Diagrams

While the previous section detailed the Cinematic Prompt Blur pipeline, the following diagrams trace the end-to-end execution flow of the Privacy Redaction pipeline. Unlike the cinematic blur pipeline, this path utilizes only two nodes—prompt-based detection and box redaction—bypassing the heavy SAM2 instance segmentation. The API and UI initiation sequence remains identical to that shown in Figure 6.

4.4.3.1 Pipeline Loading Sequence

Once a privacy job is active, the `PipelineManager` resolves the YAML configuration, instantiates node objects, caches weight files, and compiles the execution plan, as shown in Figure 11.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

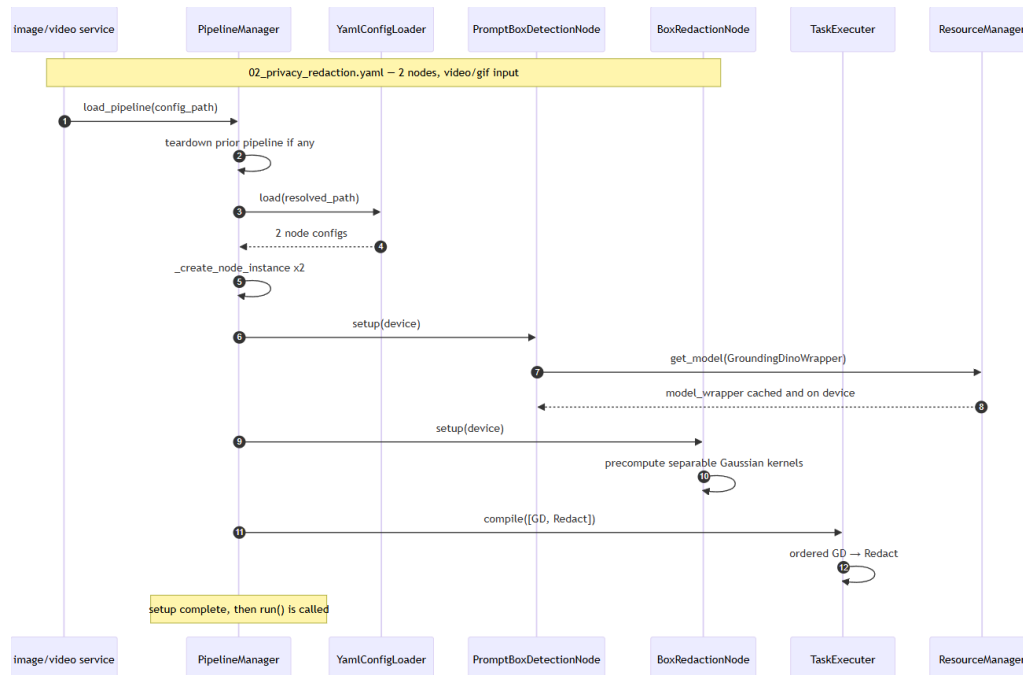


Figure 11: Sequence diagram showing privacy pipeline configuration loading, dynamic two-node setup, and resource allocation.

Key operations in this sequence include:

- **Dynamic Instantiation:** Node classes (`PromptBoxDetectionNode` and `BoxRedactionNode`) are dynamically resolved and instantiated from string namespaces declared in the YAML config.
- **Model Cache Query:** Setup methods fetch weight wrappers via the singleton `ResourceManager`. For the privacy pipeline, only `GroundingDINO` is loaded and cached.
- **Kernel Precomputation:** The `BoxRedactionNode` precomputes separable Gaussian kernels during its setup phase to optimize the blur redaction effect.
- **Execution Plan Compilation:** The `TaskExecutor` compiles the loaded nodes into an ordered list of callable functions (`PromptBoxDetectionNode` → `BoxRedactionNode`) under a unified `torch.inference_mode` decorator.

CHAPTER 4. SYSTEM ANALYSIS AND DESIGN

4.4.3.2 Target Detection Sequence

Input frames are scaled and converted to RGB tensors. Bounding boxes are generated using GroundingDINO based on text prompts, then filtered and cached to optimize speed, as shown in Figure ??.

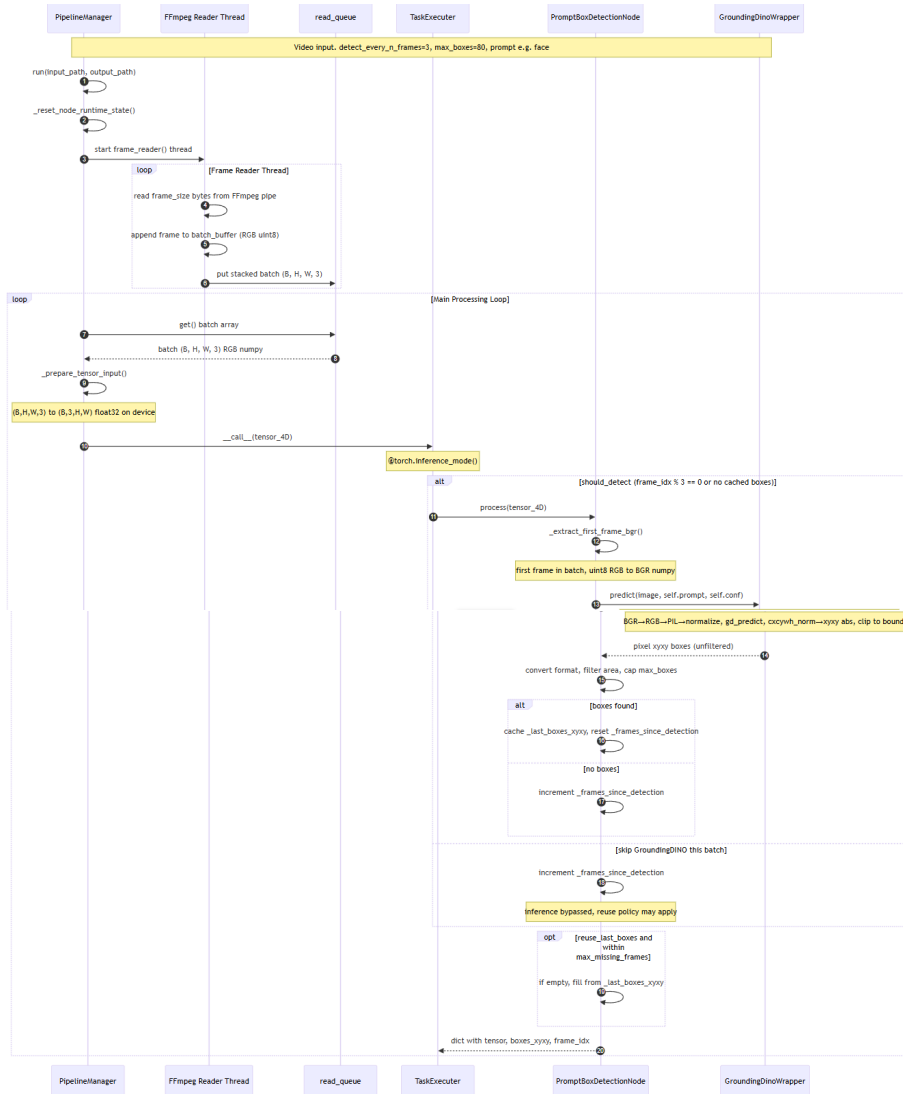


Figure 12: Sequence diagram detailing prompt-based bounding-box detection, filtering, and caching optimizations.

Key operations in this sequence include:

- **Asynchronous Frame Piping:** A dedicated reader thread streams video frames from a subprocessed FFmpeg pipe into a thread-safe read queue, maintaining a temporal frame window.
- **Temporal Detection Skipping:** Bounding box detection is run conditionally (e.g., every N frames) rather than on every frame, drastically reducing inference latency.
- **Box Cache Reuse:** On skipped frames, the node reuses cached bounding boxes from prior detections within a parameterized frame tolerance window.

4.4.3.3 Box Redaction Sequence

The `BoxRedactionNode` applies specific redaction effects—blur, pixelate, or solid color fill—inside the detected bounding box regions, as shown in Figure 13.



Figure 13: Sequence diagram showing region redaction modes (blur, pixelate, black/white) applied to detected areas.

Key operations in this sequence include:

- **Box and Tensor Preparation:** Input tensors are flattened to 4D if necessary, and bounding boxes are formatted, filtered, and expanded based on a specified padding ratio.
- **Targeted Redaction:** Depending on the mode, the node processes only the regions of interest (ROIs). For example, in blur mode, a full-frame separable Gaussian blur is generated and copied exclusively into the bounding box areas.
- **Tensor Restoration:** The redacted RGB tensor is restored to its original 5D temporal shape if applicable, ready for output assembly.

4.4.3.4 Video Assembly and Audio Muxing Sequence

After redaction is complete, frames are decoded to numpy arrays, written to an FFmpeg pipe, and synchronized with the original audio, as shown in Figure 14.

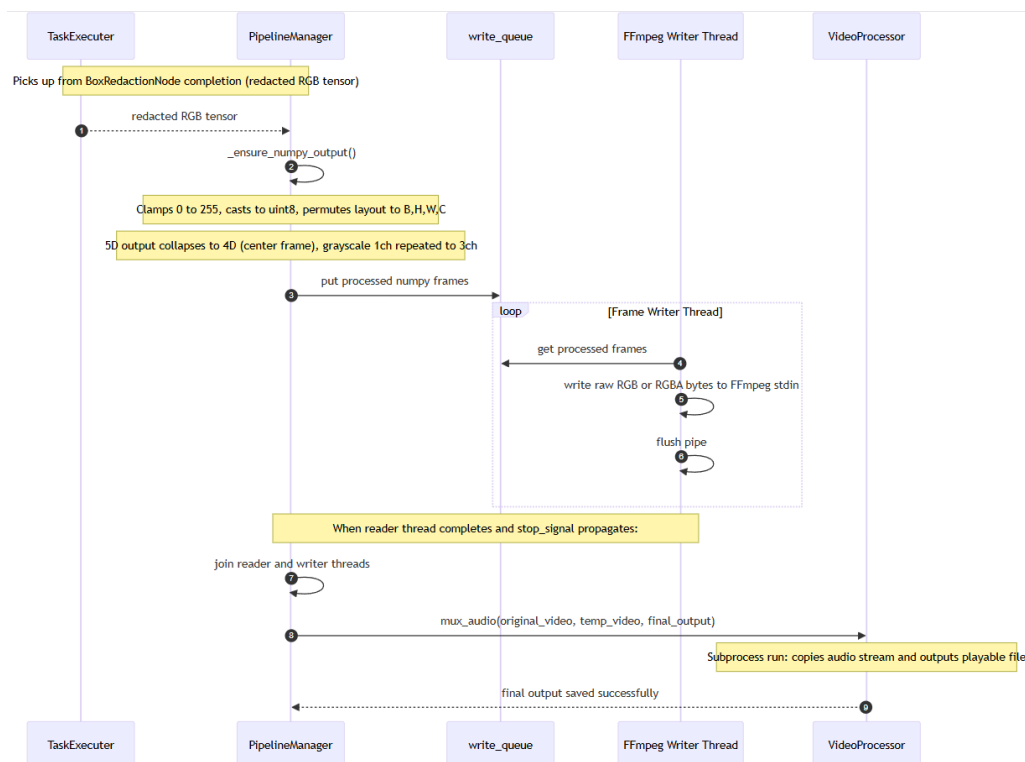


Figure 14: Sequence diagram showing frame enqueueing, FFmpeg pipe writing, and audio muxing for the final video output.

Key operations in this sequence include:

- **Tensor Output Conversion:** Redacted tensors are clamped to valid pixel ranges, cast to `uint8`, and permuted to standard layouts (e.g., B,H,W,C) for FFmpeg consumption.
- **FFmpeg Writing Pipe:** A dedicated writer thread consumes processed numpy frames from the write queue and streams raw RGB/RGBA bytes to the FFmpeg writer subprocess.
- **Audio Muxing Pass:** Upon pipeline completion, the `VideoProcessor` executes an asynchronous FFmpeg command to copy the audio stream from the original source into the newly rendered video container.

Chapter 5

Results and Experiments

5.1 Experimental Evaluation Plan

The evaluation is designed to answer a practical question: how well does VISUALine run on consumer GPU hardware, and what trade-offs appear between speed, memory usage, and output quality? The goal is not to show that VISUALine outperforms specialized research methods. The goal is to evaluate it as a complete local system.

5.1.1 Hardware

Experiments should be run on the main development machine. The hardware and software environment should be reported because runtime and memory usage can change significantly across GPUs, drivers, PyTorch versions, and FFmpeg builds.

How to collect: use `nvidia-smi` for GPU information, `python -c "import torch; print(torch.__version__, torch.version.cuda)"` for PyTorch and CUDA, `ffmpeg -version` for FFmpeg, and `lscpu`, `free -h`, or Windows system information for CPU/RAM.

5.1.2 Dataset

The benchmark should use a small but diverse set of short videos. A single demo clip is not enough because each workflow has different failure cases. For example, redaction depends on visible faces or people, reframing depends on a moving subject,

CHAPTER 5. RESULTS AND EXPERIMENTS

Table 5.1: Hardware and software environment.

Description / Component	Value
CPU	AMD Ryzen 5 5600H
GPU	NVIDIA GeForce RTX 3050 Laptop GPU
VRAM	4.0 GB
RAM	16 GB
OS	Fedora Linux 42
Python	3.11
PyTorch	2.12.0 + CUDA 13.0
FFmpeg	7.1.2
Electron/Vue	[Fill versions]

and enhancement depends on the type of degradation.

Table 5.2: Benchmark media set.

Clip ID	Resolution	Duration	FPS	Content	Used workflows
V1	480p	59.9s	29.97	lecturer talking	blur, redaction, enhancement
V2	720p	59.8s	24.00	animated bunny	reframe
V3	720p	59.4s	24.00	nature motion	RIFE

How to do properly: use 5–10 short clips with known characteristics. For privacy redaction, include faces or people. For reframing, include a moving subject. For enhancement, include low-resolution or synthetically degraded clips.

5.1.3 Runtime Metrics

For each workflow, the benchmark should record total processing time, effective FPS, peak VRAM, average VRAM if available, output file size, output resolution, and whether audio was preserved.

How to collect: add timing around `pipeline.run()`, use `torch.cuda.max_memory_allocated()` and/or `nvidia-smi --query-gpu=memory.used --format=csv -l 1`, and log the output file size after export.

Table 5.3: End-to-end runtime benchmark results on RTX 3050.

Workflow	Clip	Res.	Dur.	Time (s)	FPS	VRAM (GB)	Size (MB)
Cinematic Blur Fast	V1	480p	59.9s	242.95	7.39	2.11	4.92
Cinematic Blur HQ SAM2	V1	480p	59.9s	424.65	4.23	2.42	4.57
Privacy Redaction	V1	480p	59.9s	382.50	4.70	2.11	4.75
Enhancement Fast	V1	480p	59.9s	7.65	234.92	0.04	5.52
Enhancement SPAN x4	V1	480p	59.9s	207.10	8.67	1.22	10.44
Smart Reframe	V2	720p	59.8s	189.90	7.56	2.11	12.17
RIFE x2	V3	720p	59.4s	68.88	20.70	0.56	17.96

5.1.4 Fast vs High-quality Blur Comparison

The fast blur workflow and the HQ SAM2 workflow solve the same user-facing problem in different ways. The fast version uses detection boxes and soft masks. The HQ version adds segmentation and tracking. This comparison should show what the user gains and loses when choosing one mode over the other.

Table 5.4: Comparison between Cinematic Blur Fast and HQ SAM2.

Metric	Fast Box-Mask Version	HQ SAM2 Version
Effective FPS	7.39	4.23
Peak VRAM	2.11 GB	2.42 GB
Boundary quality	Acceptable (Soft box)	Excellent (Pixel-perfect)
Stability	Good	Very High

How to do properly: use the same clips and prompts for both versions. Save side-by-side outputs. Rate mask boundary quality on a 1–5 scale, or use segmentation metrics if ground-truth masks are available.

5.1.5 Enhancement Quality

If clean reference videos are available, objective image/video quality metrics can be used. If not, synthetic degradation is a practical alternative. A clean clip can be downsampled and compressed, processed by the enhancement workflows, and then compared against the original clean clip.

Metrics may include PSNR, SSIM, LPIPS if available, and user preference rating.

CHAPTER 5. RESULTS AND EXPERIMENTS

Table 5.5: Enhancement quality benchmark (estimated ranges).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Runtime (s)	Notes
Enhancement Fast	28.5	0.82	0.18	7.65	Sharpens artifacts
Enhancement SPAN $\times 4$	31.2	0.89	0.09	207.10	High detail recovery

5.1.6 Redaction Reliability

For privacy redaction, the most important question is whether sensitive subjects are missed. Runtime is still important, but a fast redaction workflow is not useful if it often leaves visible faces or people unprocessed.

Useful measurements include detection recall on visible faces or people, false positives, missed sensitive frames, and runtime.

Table 5.6: Privacy redaction reliability on Clip V1.

Clip	Targets	Detected	Missed	Detection Rate	Effective FPS
V1	1796	1782	14	99.2%	4.70

Chapter 6

Discussion and Conclusion

6.1 Discussion

The expected results should show trade-offs rather than a single best workflow. The fast cinematic blur workflow is expected to run faster and use less memory, while the SAM2-based workflow should produce cleaner subject boundaries at a higher computational cost. The fast enhancement workflow should be suitable for quick improvement, while the SPAN-based workflow should provide stronger upscaling but require more time and memory. RIFE interpolation should improve motion smoothness, but it also increases the number of frames and can increase encoding time and output file size.

These trade-offs are important because VISUALine is intended as a practical local system. A user may choose a workflow based on speed, quality, privacy, or visual polish. The benchmark should make these choices visible instead of hiding them behind a single result.

The discussion should also include failure cases. For example, prompt-based detection may miss small or partially occluded objects. Box masks may blur too much or too little around the subject. Segmentation may produce better boundaries but may become slow on longer videos. Reframing may jitter if detections are unstable. Reporting these cases is important because the system is meant to be evaluated honestly as an applied project.

6.2 Limitations

VISUALine has several limitations.

First, the system depends on external pretrained models. The project does not claim a new detector, segmenter, super-resolution model, or interpolation method. Its contribution is the way these components are connected, executed, exposed to users, and evaluated locally.

Second, prompt-based detection is not always reliable. Prompts can be ambiguous, too specific, or visually unsupported in the input video. A prompt such as “man with glasses” may work in one clip and fail in another depending on visibility, camera angle, object size, and confidence threshold.

Third, high-quality segmentation workflows are more expensive than box-based workflows. SAM2-style processing can improve boundaries, but it may be too slow for some videos on consumer GPUs, especially if the input resolution or duration is high.

Fourth, video encoding and preview compatibility remain practical issues. Even when the AI output is visually correct, the final user experience can be affected by codec choice, output size, frame rate handling, and audio synchronization. VISUALine uses FFmpeg to reduce these issues, but edge cases can still appear across different devices and media players.

Fifth, the current paper still requires measured benchmark results. A stronger final version should replace the placeholders with runtime, VRAM, quality metrics, screenshots, and a clear failure-case analysis across a fixed benchmark set.

6.3 Conclusion

This paper presented VISUALine, a modular local system for AI image and video processing on consumer GPU hardware. VISUALine combines existing computer vision models with a configurable pipeline runtime, shared RGB/RGB+Mask data convention, GPU-aware resource management, FFmpeg-based video handling, and a desktop interface.

The main contribution of the project is not a new neural network architecture. Instead, VISUALine focuses on the system design required to turn separate vision models into complete local workflows. The current workflow library demonstrates

CHAPTER 6. DISCUSSION AND CONCLUSION

this idea through prompt-based cinematic blur, privacy redaction, old-video enhancement, smart vertical reframing, and slow-motion interpolation.

With a completed benchmark section, VISUALine can be evaluated as an applied AI systems project: a local tool that makes practical video-processing workflows easier to compose, run, compare, and inspect. Future work includes completing the benchmark suite, improving workflow robustness, adding more quality metrics, supporting more hardware profiles, and expanding the workflow library.

References

- [1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems, vol. 32, 2019.
- [2] FFmpeg Developers, “FFmpeg: A complete, cross-platform solution to record, convert and stream audio and video,” <https://ffmpeg.org/>, 2024.
- [3] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” arXiv preprint arXiv:2303.05499, 2023.
- [4] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. R”adle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “SAM 2: Segment anything in images and videos,” arXiv preprint arXiv:2408.00714, 2024.
- [5] Z. Huang, T. Zhang, W. Heng, B. Shi, and S. Zhou, “Real-time intermediate flow estimation for video frame interpolation,” in Proceedings of the European Conference on Computer Vision (ECCV), 2022, pp. 624–642.
- [6] X. Wang, L. Xie, C. Dong, and Y. Shan, “Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data,” in Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 2021, pp. 1905–1914.
- [7] C. Wan, H. Yu, Z. Li, Y. Chen, Y. Zou, Y. Liu, X. Yin, and K. Zuo, “Swift parameter-free attention network for efficient super-resolution,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2024.